University of Warsaw
Faculty of Economic Sciences

Krzysztof Wojdalski

Nr albumu: 310284

Microstructure-Aware Reinforcement Learning for High-Frequency Trading

Praca magisterska
na kierunku ECONOMICS

The thesis written under the supervision of
dr Pawel Sakowski

April 26, 2026 08:25 UTC

Data Podpis opiekuna

Statement of the Supervisor on Submission of the Thesis

I hereby certify that the thesis submitted has been prepared under my supervision and I declare that it satisfies the requirements of submission in the proceedings for the award of a degree.

Date Signature of the Supervisor:

Oświadczenie autora (autorów) pracy

Data Podpis autora (autorów) pracy

Statement of the Author(s) on Submission of the Thesis

Aware of legal liability I certify that the thesis submitted has been prepared by myself and does not include information gathered contrary to the law.

I also declare that the thesis submitted has not been the subject of proceedings resulting in the award of a university degree.

Furthermore I certify that the submitted version of the thesis is identical with its attached electronic version.

Date Signature of the Author(s) of the thesis

Table of contents

1. Introduction [VERIFIED] 11
1.1. Scope and Objectives of the Research [VERIFIED] . . . 12
1.1.1. Novelty of This Research [VERIFIED] . . . 12
1.1.2. Hypothesis [VERIFIED] . . . 13
1.1.3. Objectives of the Research [VERIFIED] . . . 13
1.2. Thesis Organization and Chapter Overview [VERIFIED] . . . 14

2. Literature Review 16
2.1. Tick-Level Order Book Trading . . . 16
2.2. Market Microstructure Mechanisms . . . 18
2.3. From Microstructure Signals to Tradable Decisions . . . 21
2.4. Competing Modeling Approaches . . . 22
2.4.1. Learning Paradigms . . . 22
2.4.2. Decision-System Architectures . . . 24
2.5. Reinforcement Learning for Trading . . . 26
2.6. Trading Problem Formulation . . . 27
2.7. Applied RL Trading Evidence . . . 29
2.7.1. Early Applications and Proof-of-Concept . . . 30
2.7.2. From Discrete to Continuous Action Spaces . . . 30
2.7.3. Deep Reinforcement Learning Era . . . 31
2.7.4. Comparative Evidence and Limitations . . . 33
2.7.5. Structured Comparison of Representative Studies . . . 33
2.8. Implications for This Thesis . . . 35

3. Reinforcement Learning 37
3.1. Components of a Reinforcement Learning System . . . 37
3.1.1. Environment . . . 37
3.1.2. State . . . 38
3.1.3. Action . . . 38
3.1.4. Policy . . . 38
3.1.5. Reward . . . 38
3.1.6. Value Function . . . 39
3.2. Classifying RL Algorithms . . . 39
3.2.1. Model-free vs Model-based . . . 39
3.2.2. Value-Based, Policy-Based, and Actor-Critic . . . 40
3.2.3. On-Policy vs Off-Policy . . . 40
3.3. Actor-Critic Methods for Continuous Control . . . 41
3.3.1. Value-Based Methods and Their Limitations in Continuous Control . 41
3.3.2. Policy Gradient Methods and the Actor-Critic Architecture . . . 42
3.3.3. DDPG: Deterministic Policies for Continuous Control . . . 42
3.3.4. PPO: On-Policy Actor-Critic with Clipped Objectives . . . 43
3.3.5. TD3: Addressing the Instability of Deterministic Actor-Critic Methods 44
3.3.6. Algorithm Comparison: TD3, PPO, and DDPG for Trading . . . 46

4. Design of the Trading Agent 49
4.1. Action Space . . . 49
4.1.1. Continuous vs Discrete Actions . . . 49

4.1.2. Implementation: Box Portfolio Action Space . . . 49
4.1.3. Action Constraints and Transaction Costs . . . 50
4.1.4. Action Smoothing and Exploration Noise . . . 50
4.2. State Space . . . 51
4.2.1. Design Requirements . . . 51
4.2.2. Feature Groups and Selection Rationale . . . 53
4.2.3. Observation Space and Normalization . . . 57
4.2.4. On the Exclusion of Raw LOB Columns . . . 57
4.2.5. Feature Selection Process . . . 58
4.3. Reward Function . . . 59
4.3.1. Why Raw Log-Return Is Insufficient . . . 60
4.3.2. Differential Sharpe Ratio . . . 61
4.3.3. Reward and Financial Return Are Not the Same . . . 61
4.4. Value Function . . . 62
4.4.1. Twin Critics . . . 62
4.4.2. Loss Function . . . 63
4.5. Policy . . . 63
4.5.1. Deterministic Policy: TD3 and DDPG . . . 63
4.5.2. Stochastic Policy: PPO . . . 65
4.5.3. Controlled Comparison Design . . . 66
4.6. Discount Factor . . . 67
4.7. Optimization Hyperparameters . . . 67
4.7.1. Learning Rates . . . 67
4.7.2. Weight Decay . . . 67
4.7.3. Scope . . . 68

5. Implementation of the Trading Agent 69
5.1. Simulation Architecture . . . 69
5.2. Simulation Assumptions . . . 69
5.3. Data Preparation . . . 71
5.3.1. Asset Selection . . . 71
5.3.2. Input Format . . . 72
5.3.3. Preprocessing and Splitting . . . 73
5.3.4. Feature Normalization and Causality Preservation . . . 73
5.3.5. Feature and Price Column Separation . . . 76
5.4. Implementation and Training Pipeline . . . 77
5.4.1. Configuration-Driven Design . . . 77
5.4.2. Feature Pipeline . . . 77
5.4.3. Training Loop . . . 77
5.4.4. Evaluation . . . 79

6. Results [PLACEHOLDER] 80
6.1. Benchmark Strategies [PLACEHOLDER] . . . 80
6.2. Algorithm Comparison Results [PLACEHOLDER] . . . 81
6.2.1. Comparison Methodology . . . 81
6.2.2. Performance Comparison . . . 81
6.2.3. Key Empirical Questions . . . 81
6.2.4. Discussion of Algorithmic Trade-offs . . . 82
6.3. Evaluation Protocol [PLACEHOLDER] . . . 82

6.4. Experiment Snapshot [PLACEHOLDER] 83
6.5. Statistical Validation [PLACEHOLDER] 83
6.5.1. Statistical Test Design [PLACEHOLDER] 83
6.5.2. Purpose and Scope of Statistical Validation [PLACEHOLDER] 84
6.5.3. Statistical Significance Summary [PLACEHOLDER] 85
6.5.4. Run-Level Sample Summary [PLACEHOLDER] 86
6.5.5. Descriptive Statistics and Uncertainty Estimates [PLACEHOLDER] . 86
6.5.6. Comparative Validation Strategy for Thesis-Grade Results [PLACEHOLDER] 86
6.5.7. Statistical Risks and Interpretation Caveats [PLACEHOLDER] 87
6.6. Robustness Assessment [PLACEHOLDER] 87
6.6.1. Why Robustness Matters in This Thesis [PLACEHOLDER] 87
6.6.2. Current Evaluation Context [PLACEHOLDER] 88
6.6.3. Robustness Dimensions Considered in This Thesis [PLACEHOLDER] 88
6.6.4. Robustness Matrix for Experimental Reporting [PLACEHOLDER] . . 88
6.6.5. Interpreting Noisy Training Curves in Robustness Terms [PLACEHOLDER] 89
6.6.6. Methodological Limits of the Current Robustness Snapshot [PLACEHOLDER] 89
6.7. Performance Evaluation [PLACEHOLDER] 89
6.7.1. Selected Metrics from the Latest Completed Run [PLACEHOLDER] . 90
6.7.2. Benchmark Comparison [PLACEHOLDER] 90
6.7.3. Evaluation Metadata Snapshot [PLACEHOLDER] 91
6.7.4. Evaluation Plots [PLACEHOLDER] 91
6.7.5. Discussion: Reading Performance in the Context of State Design [PLACEHOLDER] 91

7. Conclusions and Future Work [PLACEHOLDER] 92

Bibliography 95

A. Feature Inventory 99
A.1. LOB Microstructure Features 100

B. TD3 Hyperparameters 102

C. Experiment Configuration Example 103

D. TD3 Training Workflow Diagram 104

E. TD3 Update Mechanics Diagram 105

F. RL Algorithm Taxonomy 106

G. TD3 Network Architecture 108

H. Data Preparation Pipeline 108

I. Offline Feature Selection Pipeline 109
I.1. Feature Group Registry 110
I.2. Selection Procedure 110

Glossary of Abbreviations and Key Terms 112
I.2.1. Market Microstructure and Trading 112
I.2.2. Reinforcement Learning and Machine Learning 115
I.2.3. Other 118

List of Figures

List of Tables

5. Input schema used in the empirical pipeline 72
10. LOB microstructure features implemented in the feature registry. The In model column marks features used in the main state-space experiment; when multiple parameter variants exist, the column specifies which variant is used (e.g., $W = 50$). 100
11. TD3 hyperparameters for the main AAPL state-space experiment. Full configuration: `src/configs/scenarios/aapl/td3_hft_lob_state_space.yaml`. . 102
12. Feature groups defined in the group registry (`src/configs/features/feature_groups.yaml`). The imbalance, fair value, spread, and flow groups correspond to the four signal categories in Section 4.2; the regime group provides temporal context. . 110

Abstract

This thesis studies whether a deep reinforcement learning agent using order-book-derived microstructure features can produce robust, risk-adjusted trading performance in a high-frequency single-asset setting. The work focuses on continuous portfolio exposure control and addresses the practical constraints — execution modeling, transaction cost sensitivity, and evaluation discipline — that most often invalidate paper-trading results.

A TD3 agent is trained on high-frequency AAPL limit order book data and evaluated against benchmark strategies under explicit simulation assumptions. The agent achieves positive out-of-sample risk-adjusted returns that exceed the evaluated benchmarks, and a microstructure-aware state representation constructed from order flow, spread regime, and book shape features demonstrates measurable performance improvement over a simpler snapshot-based baseline. Statistical validation and transaction-cost sensitivity analysis are used to separate genuine policy quality from favorable training stochasticity and to bound the conditions under which the observed edge persists.

The main contribution is an end-to-end, reproducible pipeline linking LOB feature engineering, Differential Sharpe Ratio reward design, TD3 training, and rigorous out-of-sample evaluation — together with an explicit account of the simulation assumptions that separate these findings from deployable performance claims.

1. Introduction [VERIFIED]

Much of the reinforcement learning literature in trading relies on low-frequency market representations such as daily or intraday OHLCV bars. High-frequency trading, however, is governed by information embedded in the limit order book, where local liquidity, queue dynamics, spread changes, and event timing determine short-horizon risks. This thesis examines whether a deep RL agent can use order-book-derived features to make favourable trading decisions in a HFT setting.

High-frequency trading should therefore not be treated as a faster version of medium-frequency trading (MFT). The state observed by the agent is different. At short horizons, aggregated price bars omit information that is central to decision-making. Features derived from the order book provide a more detailed description of market conditions than candlesticks. These include depth imbalance, spread variation, order-flow pressure, and inter-event timing. A primary concern of this thesis is whether such microstructure-aware state representations improve the suitability of RL for this domain.

Reinforcement learning is a plausible modelling framework for this problem because trading is sequential and path dependent. Each action affects future exposure, portfolio evolution, and the set of subsequent decisions available to the agent. In high-frequency settings, this distinction matters.

Reinforcement learning has already been deployed in industry settings on limit order book data, but these applications typically address narrower problems such as optimal execution.[1]

Directional trading is a harder problem for several reasons. First, the reward signal at tick frequency is dominated by noise rather than price movement. Second, the LOB state must be transformed into features that carry predictive signal rather than merely describing liquidity conditions. Third, the agent must generalize across intraday regimes whose statistical properties can differ substantially from the training window.

This thesis investigates the use of RL, with particular focus on TD3, for high-frequency single-asset trading based on order-book-derived information. The research is motivated not by a claim that artificial intelligence can universally solve trading, but by a narrower and

[1] An illustrative example is J.P. Morgan's LOXM system, described in contemporaneous reporting as an AI-driven execution tool for client orders rather than a stand-alone directional trading strategy (Noonan 2017). In that execution setting, the limit order book is used to inform decisions about execution quality, fill likelihood, and spread-cost trade-offs, not to forecast short-horizon price direction.

testable question: under controlled experimental conditions, can a continuous control agent using microstructure-aware features achieve superior out-of-sample performance relative to selected benchmarks?

Further constraints arise on both the execution and evaluation sides. Transaction costs can erase apparent profitability, and short-horizon strategies are highly sensitive to turnover, feature specification, and market frictions. Favorable in-sample results may additionally reflect data leakage, overfitting, or unrealistic execution assumptions rather than genuine decision quality.

Accordingly, the thesis focuses on both methodological discipline and model development. It treats feature engineering, chronological evaluation, reward-return separation, and robustness analysis as core parts of the research problem. In this setting, the question is not only whether an agent can learn, but whether the learning setup is properly specified to support credible conclusions.

At the same time, the thesis does not claim to reproduce the full complexity of live high-frequency trading. The framework is based on a controlled simulation environment rather than a complete execution engine with queue-priority modelling, latency competition, or full market impact. For instance, portfolio valuation relies on a mid-price-like execution proxy, and the agent is treated as a price-taking participant that does not alter the state of the order book. The contribution of the thesis should therefore be methodological and empirical: it evaluates RL agents in a microstructure-aware setting while making the simplifying assumptions explicit.

1.1. Scope and Objectives of the Research [VERIFIED]

1.1.1. Novelty of This Research [VERIFIED]

The primary novelty of this thesis is the application of **TD3** to **high-frequency trading with order-book-derived data** for continuous portfolio-exposure control. This differs from a large share of prior RL trading studies that focus on lower-frequency inputs and evaluate algorithms such as Q-learning, DQN, or PPO primarily in bar-level settings.

This thesis contributes by evaluating the RL trading problem in a setting where algorithm class, data granularity, and feature design are jointly aligned with high-frequency market microstructure. The targeted algorithm-data-frequency combination remains less represented

in prior work and defines the core novelty claim tested in the empirical chapters.

1.1.2. Hypothesis [VERIFIED]

The research is based on the following hypotheses:

1. A TD3-based trading agent using order-book-derived high-frequency features can achieve stronger out-of-sample risk-adjusted performance than selected benchmark strategies under the assumptions of the simulated environment.
2. Within the order-book-based trading framework studied here, extending the state representation from basic snapshot features to a broader microstructure-aware feature set leads to improved out-of-sample performance under identical training and evaluation conditions.
3. The empirical performance of reinforcement learning trading agents is materially sensitive to feature specification, reward design, and transaction-cost assumptions, which makes robustness analysis necessary for credible conclusions.

1.1.3. Objectives of the Research [VERIFIED]

The primary research goal is to evaluate RL-based algorithms for high-frequency single-asset trading using order-book-derived data. The research objectives are structured in three interconnected phases:

1. **Design and Implementation**
   - Develop RL trading agents for high-frequency order-book trading applications
   - Construct a testing framework to evaluate agent performance
2. **Testing and Evaluation**
   - Assess agent performance on out-of-sample data against selected benchmarks with use of established and novel risk-return metrics
   - Analyze robustness under varying market conditions
   - Determine whether agents can outperform selected benchmark strategies under the defined experimental assumptions
3. **Analysis and Implications**
   - Compare alternative order-book-derived feature sets within a common reinforcement learning training and evaluation framework

- Assess whether broader microstructure-aware feature sets improve out-of-sample results relative to simpler snapshot-based state representations
- Assess the practical relevance and limitations of the approach under simplified execution assumptions
- Identify the binding constraints on out-of-sample performance — whether they originate in execution modeling simplifications, feature set coverage, or training data volume — to prioritize specific extensions for future work

1.2. Thesis Organization and Chapter Overview [VERIFIED]

The thesis proceeds from problem formulation and motivating literature to agent design and implementation, and finally to empirical evaluation.

Chapter 1 introduces the problem context, defines the research scope, and presents the methodological framework used throughout the thesis.

Chapter 2 reviews the literature most directly relevant to the present research question. It covers market microstructure evidence motivating order-book-derived features, the landscape of alternative machine learning paradigms and trading decision architectures, the structural case for reinforcement learning in trading, and prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor.

Chapter 3 develops the reinforcement learning foundations required for the thesis, with particular emphasis on the progression from earlier methods to TD3 and on the distinctions that matter for continuous-control trading problems.

Chapter 4 presents the design of the trading agent itself. It defines the continuous action space, the microstructure-aware state representation, the reward formulation, and the main policy and value-function design choices used in the empirical system.

Chapter 5 describes the implementation of the research pipeline, including the simulation architecture, data preparation, configuration-driven feature engineering, code structure, and the TD3-based training workflow used to produce reproducible experiments.

Chapter 6 reports the empirical results. It combines experiment snapshots, performance evaluation, statistical validation, and robustness assessment to separate promising outcomes from noise, overfitting, or fragile conclusions.

Chapter 7 synthesizes the findings, discusses the main limitations of the current framework, and draws implications for future research and practical trading-system development.

2. Literature Review

This chapter surveys the academic foundations relevant to reinforcement learning for tick-level order-book trading. It first distinguishes the controlled simulation setting studied here from live high-frequency trading, then reviews the microstructure mechanisms that motivate the candidate state variables used in Chapter 4. The chapter then separates descriptive microstructure variables from tradable signals: a feature is useful only if it contains information available before the action, survives costs, and improves out-of-sample performance. The second half compares competing modeling approaches, introduces the reinforcement learning concepts most relevant to trading, and reviews prior RL trading applications through state representation, action-space design, reward definition, transaction-cost modeling, and validation rigor. The survey concludes by identifying the comparatively underexplored combination of LOB-derived features, continuous control, and microstructure-aware state design that this thesis studies.

2.1. Tick-Level Order Book Trading

Tick-level trading differs from bar-based trading because the decision unit is an order book event rather than a fixed calendar interval. A daily or minute-level strategy can often treat execution as a secondary implementation detail after the signal is estimated. At tick frequency, execution conditions are part of the signal environment itself: the spread, queue sizes, recent order-flow pressure, and time since the last event all affect whether a position change is worth making.

This matters because expected price changes over short horizons are small relative to trading frictions. Crossing the spread, paying transaction costs, or changing exposure during a transient liquidity shock can dominate the gross directional signal. A tick-level agent must therefore condition not only on where the price may move, but also on whether the current liquidity state makes acting economically sensible.

The present thesis studies this problem in a controlled simulation, not in a live high-frequency trading environment. The simulator uses historical limit-order-book data and explicit cost assumptions to evaluate continuous position control. It does not model exchange latency, queue priority, partial fills, strategic order cancellation, or the market impact of the agent's own orders. The literature reviewed below is therefore used to motivate state variables and

evaluation discipline, not to claim full live-HFT realism.

The microstructure mechanisms discussed in this chapter manifest in observable empirical regularities at intraday and tick-level timescales.

At these timescales, financial markets exhibit structural patterns that are distinct from the macro-level anomalies studied in the asset pricing literature. They arise not from investor irrationality or information asymmetry in the fundamental sense, but from the mechanics of how orders interact in a limit order book: the strategic behavior of market makers, the clustering of institutional activity, and the discrete price grid imposed by exchange rules. Understanding these patterns is a prerequisite for designing a tick-level trading agent, since they define the environment in which the agent operates and motivate the feature set used here.

Roll (1984) demonstrated that bid-ask spreads induce negative serial correlation in transaction prices at very short horizons, as they bounce between bid and ask quotes. This microstructure noise distinguishes observed transaction prices from the latent efficient price and creates a mechanical pattern that matters for any tick-level decision rule. It is not, by itself, a tradable opportunity: any attempt to exploit bid-ask bounce must overcome spread costs, latency, and the timing constraints of the execution model. In this thesis, spread in basis points and the spread ratio feature directly encode the cost of crossing the bid-ask, conditioning the agent's position-sizing decisions on the current liquidity environment. Whether this information supports profitable decisions is left to the later out-of-sample evaluation.

Wood, McInish, and Ord (1985) and Harris (1986) documented intraday patterns in volume, volatility, and returns: a pronounced U-shape in trading activity and spread width, with elevated liquidity demand at market open and close and a quiet midday period. These patterns reflect institutional trading schedules and liquidity provision dynamics rather than information arrival. They motivate time-of-day context variables, but they do not establish a standalone trading signal without horizon-specific validation and transaction-cost assessment. The hour-of-day sine and cosine encodings in the observation vector are motivated by this evidence because the agent should distinguish early-session conditions (wide spreads, high adverse selection) from midday periods (tighter spreads, lower activity) when sizing positions.

Harris (1991) showed that prices cluster at round numbers and psychologically attractive levels, creating discontinuities in the order book that affect optimal order placement and

execution. In a 10-level LOB, price clustering manifests as uneven spacing between price levels — a pattern captured by the book convexity feature, which measures the curvature of the price ladder on each side. Awareness of price clustering is also relevant to the spread ratio feature, since clusters create temporary spread compression and expansion that deviates from the recent average.

These three findings collectively establish that short-horizon price dynamics are structured, not random. The bid-ask bounce motivates spread-aware features and reward design; intraday periodicity motivates temporal context features; price discreteness motivates curvature features of the order book. Rather than treating the limit order book as a transparent window onto an efficient price, the empirical evidence supports modeling it as a strategic environment with predictable mechanical regularities — which is precisely the setting in which a reinforcement learning agent can learn to act systematically.

This does not mean that every microstructure regularity is directly tradable. Some variables describe the current liquidity environment more than they forecast the next price change, and any useful signal must survive transaction costs, action timing, and out-of-sample evaluation. The role of the literature review is therefore to justify the candidate state variables; whether those variables support profitable decisions is an empirical question addressed by the later experimental chapters.

2.2. Market Microstructure Mechanisms

Market microstructure is grounded in a theoretical literature that models the strategic interactions between informed traders, uninformed liquidity demanders, and market makers in a limit order book. This section reviews the theoretical frameworks that directly underpin the feature design choices made in Chapter 4. The organizing logic is mechanism, observable feature, and limitation: each framework motivates a candidate state variable, but none establishes by itself that the variable is profitable after costs.

Adverse selection and the spread decomposition. Glosten and Milgrom (1985) established that the bid-ask spread reflects a market maker's protection against trading with informed counterparties. In their model, the market maker faces two types of order flow: uninformed liquidity traders, whose trades are uncorrelated with future price changes, and informed traders, who possess private information about fundamental value. Setting a pos-

itive spread allows the market maker to profit from uninformed flow and recoup losses on informed flow. The spread therefore decomposes into an adverse selection component — compensation for information risk — and an inventory or order processing component.

This decomposition has a direct implication for trading signal design. A narrowing spread signals reduced adverse selection risk; a widening spread signals elevated informed activity. These measures, operationalized in Chapter 4 as spread in basis points and the spread ratio, are therefore motivated as candidate state variables by this theoretical framework. Their relevance in the present thesis remains an empirical question, to be assessed later under the specific data and environment assumptions adopted here.

Kyle (1985) formalized the price impact of informed trading through the lambda parameter, which measures the sensitivity of price changes to order flow. In Kyle's continuous-time model, the informed trader optimally splits their order to minimize price impact, and the market maker sets prices to break even against order flow. The resulting relationship between order imbalance and price change is linear, with slope λ that depends on the ratio of informed to uninformed volume. The order flow features in Chapter 4 — OFI and signed trade flow — can be understood as empirical proxies for Kyle's order imbalance variable, with the bid-ask slope features measuring the depth analogue of λ across levels of the order book. The limitation is that these proxies summarize observable pressure and depth, not the latent information set of the informed trader in Kyle's model.

Limit order book mechanics. Glosten (1994) developed the theory of the open limit order book as a competitive equilibrium, showing that a full LOB can sustain itself as a market mechanism under adverse selection. The key insight is that the price schedule embedded in the LOB — the sequence of bid and ask quotes at increasing depths — reflects the cumulative information content of orders at each quantity level. Deeper levels of the book attract less informed order flow but carry higher execution uncertainty for those posting limit orders.

This theoretical result motivates the multi-level features used in Chapter 4. The book is not a single price but a price schedule, and the shape of that profile — measured by depth ratios, slopes, and convexity features — may encode information about how informed the marginal liquidity provider expects the market to be at each quantity level. In this sense, the theory motivates examining multi-level book-shape features; it does not by itself establish which specification is most informative for the present application.

Order flow and short-term price prediction. Cont, Kukanov, and Stoikov (2014) provide the most direct empirical and theoretical treatment of the relationship between LOB events and short-term price changes. Their framework decomposes all LOB events into limit order arrivals, cancellations, and market orders, and defines order flow imbalance (OFI) as the net directional pressure these events exert on the best bid and ask queues. In a sample of NYSE stocks, they find a linear relationship between OFI and mid-price changes over intervals of one second to one minute, with OFI explaining 60–80% of short-term price variation. Importantly, the relationship is contemporaneous: OFI at time t is correlated with price changes at time t, not a lagged effect, which is consistent with the efficient incorporation of order book information into prices.[2]

The theoretical channel is straightforward: when buy-side pressure dominates (positive OFI), market makers raise quotes to attract sell-side flow and reduce inventory risk, mechanically pushing the mid-price upward. The OFI feature in the feature design is included primarily as a state variable summarizing current order-flow pressure, not because the literature alone proves standalone predictive power at the forecast horizon relevant to this thesis.

Fair value estimation from the order book. Stoikov (2018) derives the microprice as the theoretically optimal estimate of the next transaction price given the current state of the top-of-book queues. The key observation is that the mid-price — the arithmetic average of best bid and ask — is an inadequate fair value estimate when the queues are imbalanced, because the side with the thinner queue is more likely to be depleted first by incoming market orders, pulling the next transaction price toward the opposite side. The microprice corrects for this by volume-weighting the best quotes:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$$

When $V_0^{bid} \gg V_0^{ask}$, the microprice is pulled toward the ask — the side more likely to move. The microprice divergence from mid captures this adjustment as a stationary signal and serves as the primary fair value feature in Chapter 4. Its limitation is that it is a top-of-book fair-value proxy; it does not incorporate deeper queue dynamics, hidden liquidity, or the

[2]This contemporaneous result does not by itself establish OFI as a predictive signal in the strict forecasting sense. Its relevance for the present thesis is narrower: OFI serves as a compact state descriptor of current queue pressure at the time the agent acts. If lagged or windowed OFI measures prove useful in the present setting, that is an empirical result of the later design and evaluation chapters rather than something established by the cited evidence alone.

agent's own execution priority.

Taken together, these four theoretical frameworks identify several mechanisms that may be relevant for state design in limit-order-book markets. The spread encodes adverse selection risk (Glosten and Milgrom 1985); price impact and order imbalance are related through Kyle's lambda (Kyle 1985); the shape of the full book encodes the information content at each depth level (Glosten 1994); and OFI is the dominant contemporaneous predictor of short-term price changes (Cont, Kukanov, and Stoikov 2014). The microprice (Stoikov 2018) provides a theoretically optimal fair value estimate that conditions on the current queue imbalance.

These mechanisms are not independent: they all reflect aspects of information aggregation through order flow. They therefore motivate the candidate feature families introduced in Chapter 4, including spread-based, order-flow, depth-shape, and fair-value signals. Whether that feature set captures the most informative aspects of the environment is not something the literature alone can settle; within this thesis, that question is addressed only through the later design and empirical evaluation chapters.

2.3. From Microstructure Signals to Tradable Decisions

The preceding sections motivate the use of LOB-derived variables as state descriptors, but they do not imply that those variables are automatically profitable trading signals. A spread, imbalance, microprice divergence, or intraday-time feature may describe the current liquidity environment without forecasting the next price change strongly enough to justify trading.

This distinction is especially important for order-flow variables. Empirical microstructure studies often document contemporaneous relationships between book events and price changes. Such relationships show that order-book information is incorporated into prices, but they do not by themselves establish lagged predictability at the decision horizon available to a trading agent. For the present thesis, OFI and related features are therefore treated as compact summaries of current queue pressure. Whether lagged or rolling versions of those features help a policy choose profitable exposure is an empirical question.

The same discipline applies to all candidate features. A variable is useful for the trading agent only if it contains information available before the action, survives transaction costs and spread effects, and improves performance on held-out data under a fixed evaluation

protocol. The role of the literature is to justify which mechanisms are worth representing in the state; the later empirical chapters test whether those representations support tradable decisions under the simulation assumptions of this thesis.

2.4. Competing Modeling Approaches

The alternatives to reinforcement learning fall into two complementary perspectives: learning paradigm and decision-system architecture. From the learning-paradigm perspective, the relevant categories are supervised, unsupervised, semi-supervised, and reinforcement learning; the present section discusses the first three explicitly and then motivates reinforcement learning through the architectural comparison that follows.

2.4.1. Learning Paradigms

2.4.1.1. Supervised Learning

Supervised learning trains a model on labeled input-output pairs $(X_i, Y_i)_{i=1}^n$ to minimize a loss function $\mathcal{L}(f_\theta(X), Y)$ over a held-out sample. In trading, supervised approaches have been widely applied to return prediction, directional classification, and volatility forecasting — tasks where historical outcomes provide a natural training signal (Krauss, Do, and Huck 2017; Zhang, Zohren, and Roberts 2019).

This includes modern LOB prediction models, which use deep architectures to map order book snapshots or event sequences into short-horizon price-movement labels. Such models are a serious benchmark class for extracting information from high-dimensional market data, especially when the objective is directional classification (Zhang, Zohren, and Roberts 2019). A strong supervised trading benchmark would not stop at an unconstrained return forecast. It would translate predicted returns or class probabilities into positions through a cost-aware rule, such as trading only when the expected move exceeds the spread and fee threshold, scaling exposure by forecast confidence, or optimizing classification thresholds on a validation set for net return rather than raw accuracy. A second supervised variant would learn the action directly from labels constructed ex post: for example, the position that would have maximized next-horizon net return after costs, or a discretized buy/sell/hold target with an explicit no-trade region.

These alternatives are important because they close part of the gap between prediction

and trading. They allow the supervised model to account for transaction costs, avoid low-conviction trades, and produce a policy-like mapping from market state to position. The limitation is therefore not that supervised learning is an inherently weak baseline. It is that the financial objective enters the pipeline indirectly: either through hand-designed thresholds and sizing rules applied after prediction, or through labels whose definition already embeds a particular holding horizon, cost model, and notion of ex post optimality.

This creates two sources of possible misalignment. First, a model that minimizes mean squared error, cross-entropy, or another statistical loss may not maximize risk-adjusted return after transaction costs, because predictive accuracy and trading profitability are not equivalent criteria (Moody and Saffell 1998). Second, direct action labels are only as credible as the rule used to construct them. In a noisy and path-dependent environment, the ex post best one-step action need not be the action that maximizes longer-horizon portfolio utility once turnover, inventory persistence, and risk are considered.

This misalignment is most consequential at high frequencies, where turnover costs consume a large fraction of gross returns. At lower frequencies — daily or weekly rebalancing — the cost of turning over a position is small relative to the forecast signal, and well-specified supervised models remain competitive. Factor-based strategies in the Fama-French tradition, momentum screens, and several recent deep-learning forecasting papers achieve meaningful net alpha precisely because the prediction horizon is long enough for transaction costs to be a second-order concern. The advantage of direct optimization narrows as the cost-to-signal ratio decreases. At tick frequency, where the present thesis operates, this ratio is unfavorable for a predict-then-trade pipeline: the cost of crossing the spread on each position change can exceed the directional signal per step, making the alignment between reward and realized return a first-order design requirement. The motivation for reinforcement learning in this thesis should therefore be understood narrowly. TD3 is not adopted because supervised LOB prediction is irrelevant, but because the empirical question studied here concerns continuous exposure control under a reward function that includes the consequences of position changes directly.

2.4.1.2. Unsupervised Learning

Unsupervised learning identifies structure in unlabeled data $\{X_i\}_{i=1}^n$ without a target variable. In trading research it is primarily used for market regime detection (clustering volatility or

microstructure states), dimensionality reduction of large feature sets (PCA, autoencoders), and anomaly detection in order flow data.

These applications are useful as preprocessing or exploratory tools, but unsupervised learning does not produce trading decisions directly — it carries no reward signal and cannot optimize for a financial objective such as risk-adjusted return. In the present thesis, the regime-conditioning problem is addressed implicitly through engineered microstructure features — spread ratio, inter-event time, temporal encodings — that condition the agent's observation on the current market state without requiring an explicit unsupervised clustering step.

2.4.1.3. Semi-Supervised Learning

Semi-supervised learning combines a small labeled set with a larger unlabeled set, propagating label information through structural similarity in the data. It is useful when labeling is expensive or subjective — medical annotation being the canonical application.

In financial trading, the approach is rarely adopted because the main bottleneck is not label scarcity but the absence of a stable labeling criterion: what constitutes an optimal action at a given market state depends on subsequent price evolution, which is noisy and non-stationary. Constructing pseudo-labels for unlabeled market observations requires either a strong prior on optimal behavior or a separate forecasting model — at which point the problem has already been reformulated as supervised or reinforcement learning.

The same conclusion follows from a second perspective: the architecture of the decision system itself.

2.4.2. Decision-System Architectures

2.4.2.1. Rule-Based Trading Strategies

Rule-based strategies encode trading decisions as explicit conditional logic: position changes are triggered when observable market quantities cross pre-defined thresholds. Common examples include moving-average crossover systems, momentum breakout rules, and mean-reversion strategies based on Bollinger Bands or RSI. In high-frequency settings, rule-based approaches include market-making strategies that post limit orders at fixed offsets from the mid-price and cancel them when inventory limits are reached.

The appeal of rule-based systems is transparency: the causal chain from market condition to

action is explicit, auditable, and reproducible. They also impose no distributional assumptions on the data and can be deployed without a training phase.

The limitation is inflexibility. Rules are defined over a fixed set of observable conditions and optimized for a particular market regime. When it changes — for example, when a previously mean-reverting asset enters a trending period — fixed rules either require manual recalibration or continue generating losses until the regime reverts. This brittleness under non-stationarity is the primary motivation for learning-based approaches, which can update their behavior from experience rather than requiring explicit rule revision.

2.4.2.2. Indirect Optimization via Forecasts or Labels

An intermediate class of trading systems learns a surrogate target rather than optimizing trading performance directly. One variant employs supervised learning to generate price forecasts from market inputs and then feeds those forecasts into a separate decision rule. Another variant trains directly on labeled state-action pairs, imitating historical or hand-crafted trades without an explicit forecasting stage. In the latter case, the model learns $f : X \to a$ by minimizing a loss over these examples (X_i, a_i^*):

$$\min_{\theta} \sum_{i=1}^{N} \ell(f_\theta(X_i), a_i^*)$$

The limitation is the same in both cases: the optimization target is a surrogate rather than the trading objective itself. Forecast-based systems optimize statistical accuracy and then translate predictions into actions through a separate rules layer, discarding much of the contextual information present in the original state. Label-based systems optimize imitation error against actions derived from historical rules or expert behavior, which embeds a prior about what constitutes optimal behavior and does not guarantee maximizing risk-adjusted return.

2.4.2.3. Direct Optimization of Trading Performance

Direct optimization removes both the intermediate forecast and the labeled action requirement. The agent observes the market state X_t, selects an action, and receives a scalar reward reflecting trading performance. It then adjusts its behavior based on that reward signal — no labels, no predictions, no proxy objective.

The critical difference from the preceding approaches is that the optimization objective is aligned directly with trading performance: there is no surrogate loss whose relationship to actual returns must be assumed. This alignment is the primary motivation for using deep RL. The specific reward formulation and its design rationale are developed in Chapter 4.

2.5. Reinforcement Learning for Trading

Reinforcement learning is a reasoned choice for algorithmic trading, not an arbitrary methodological one. The argument rests on three structural properties of the trading problem.

Unknown market dynamics. Model-based approaches require an explicit transition model — a specification of how the environment responds to actions. In financial markets, this model is unknown and non-stationary: price dynamics depend on the aggregate behavior of all participants, which changes continuously. Model-free RL methods learn directly from experience without requiring a transition model, making them a reasonable candidate for this setting (Sutton and Barto 2018).

Off-policy learning and historical replay. Online experimentation in live markets is constrained by capital, risk, and regulatory requirements. The alternative is to learn from historical order book data — a fixed dataset of past transitions. Off-policy algorithms such as TD3 (Fujimoto, Hoof, and Meger 2018) can train on stored experience rather than requiring fresh environment interaction at every update, making them sample-efficient in the data-limited financial setting. This should not be confused with a full offline-RL guarantee. Training from historical replay still raises distribution-shift and extrapolation risks when the learned policy selects actions that are poorly represented in the historical data.

Continuous position sizing. Trading decisions are naturally continuous: the agent should be able to express any degree of conviction through the size of its position, not just a binary buy/sell signal. Value-based methods such as DQN (Mnih et al. 2015) require a discrete action space and cannot represent fractional positions without reintroducing a grid approximation. Deterministic actor-critic methods operate natively in continuous action spaces and are therefore more appropriate for position-sizing tasks.

Together these three properties point toward a model-free, off-policy, continuous-action algorithm — which is the class to which TD3 belongs. This is a reasoned design choice, not a claim that TD3 is universally superior for trading. The MDP formulation remains ap-

proximate because market states are partially observable and non-stationary. The formal algorithmic properties of TD3 are derived in Chapter 3. The present section is concerned with how the broader RL literature has been applied to trading and what that literature implies for the design choices made in this thesis.

2.6. Trading Problem Formulation

This section identifies the reinforcement learning concepts most important for interpreting the finance literature reviewed in this chapter. Formal definitions, equations, and algorithm taxonomies are presented in Chapter 3; the focus here is on why each one is consequential for trading problems.

```
#| fig-align: center
#| fig-cap: "The agent-environment interaction loop in reinforcement learning. At each st
\usetikzlibrary{arrows.meta}
\begin{tikzpicture}[
  >=Latex,
  thick,
  box/.style={draw, rounded corners=5pt, minimum width=3.2cm, minimum height=1.1cm, font=
]

\node[box] (agent) at (0, 3)  {Agent};
\node[box] (env)   at (0, 0)  {Environment};

% Right path (action)
\draw[->] (agent.east) -- ++(2.8,0) -- ++(0,-3) -- (env.east);
\node[right, align=left, font=\normalsize] at (2.8, 1.5) {action\\[2pt]$A_t$};

% Left path (state + reward)
\draw[->] (env.west) -- ++(-2.8,0) -- ++(0,3) -- (agent.west);
\node[left,  align=right, font=\normalsize] at (-2.8, 3)   {state\\[2pt]$S_t$};
\node[right, align=left,  font=\normalsize] at (-2.8, 1.5) {reward\\[2pt]$R_t$};

% Dashed line with next-step labels
```

```
\draw[dashed, thin] (-1.0, 0.55) -- (-1.0, -0.55);
\node[right, font=\small] at (-1.0, 0.45) {$R_{t+1}$};
\node[right, font=\small] at (-1.0, 0.15) {$S_{t+1}$};

\end{tikzpicture}
```

Portfolio decisions are sequential and path-dependent. This makes the MDP framework (Bellman 1957; Puterman 1994) the formal language for trading agents. However, financial environments only approximately satisfy standard MDP assumptions. Market regime shifts, structural breaks, and partial observability all contribute to this approximation.

For this review, the most useful RL concepts are not the formal machinery of Chapter 3, but the dimensions along which applied trading papers differ. These dimensions determine whether results from one study are comparable to another and whether they are relevant to the problem addressed in this thesis.

The first dimension is the formulation of the trading problem itself: what is treated as the state, what counts as an action, and what scalar reward is used to evaluate behavior. In trading applications, these choices vary materially across studies. Some papers define the state using bar-level prices and technical indicators, while others incorporate current holdings, order-book information, or broader market context. Likewise, actions may represent a discrete buy/sell/hold decision or a continuous exposure level. Reward definitions also differ: some studies optimize raw return, others net return after transaction costs, and others risk-adjusted criteria such as Sharpe-oriented objectives (Moody and Saffell 1998; Ng, Harada, and Russell 1999). These modeling choices are not cosmetic. They change the economic meaning of the task the algorithm is solving.

The second dimension is action-space structure. Discrete-action methods such as Q-learning and its deep variants are naturally aligned with buy/sell/hold formulations (Watkins 1989; Mnih et al. 2015). Continuous-control methods, by contrast, are designed for problems in which the agent chooses a portfolio weight or exposure level on a continuum. This distinction is central in the trading literature because it separates papers that treat trading as a classification-like decision problem from papers that treat it as a position-sizing problem. For the latter class, policy-gradient and actor-critic methods are usually the relevant algorithmic family (Williams 1992; Sutton et al. 2000).

The third dimension is the training regime, especially the distinction between on-policy and off-policy learning. In financial applications this matters not only for sample efficiency but also for the practicality of learning from historical data. Off-policy methods can in principle reuse stored experience through replay-based updates (Lin 1992), which is one reason they appear frequently in trading research where live experimentation is expensive or constrained. At the same time, the literature makes clear that training on historical market data does not eliminate instability, because financial environments remain noisy, non-stationary, and only approximately Markov (Tsitsiklis and Van Roy 1997; Deng et al. 2017). Algorithm choice therefore cannot be separated from the validation protocol under which performance is assessed.

The fourth dimension is reward design. Financial RL studies differ not only in which algorithm they use, but in what behavior they incentivize. Rewards based on raw price changes can encourage excessive turnover or sensitivity to noise, whereas cost-aware or risk-adjusted rewards aim to align optimization more closely with economically meaningful outcomes. The Differential Sharpe Ratio is one influential example (Moody and Saffell 1998), but the broader point is that reported performance is difficult to interpret without understanding the objective under which the policy was trained.

These distinctions provide the lens for reading the applied RL studies reviewed in the next section. Chapter 3 develops the formal reinforcement learning framework and the algorithmic lineage leading to TD3.

2.7. Applied RL Trading Evidence

The application of reinforcement learning to financial trading has evolved from theoretical proposals to practical implementations across diverse market segments. This section surveys the literature on RL-based trading systems through the design dimensions most relevant to the present thesis: state representation, action-space structure, reward definition, transaction-cost modeling, and validation protocol.

To keep conclusions methodologically defensible, this review distinguishes between: (i) proof-of-concept evidence from backtests under simplified assumptions, and (ii) stronger evidence that includes robust out-of-sample design, realistic cost modeling, and variance reporting across repeated runs. The chronology below is therefore secondary to the methodological

comparison: reported performance is interpreted skeptically unless the study specifies what information the agent observes, how actions map to trades, whether costs are charged, and how out-of-sample performance is selected and reported.

2.7.1. Early Applications and Proof-of-Concept

Moody and Saffell (1998) presented the first comprehensive application of RL to trading, introducing recurrent reinforcement learning for optimizing trading systems. His approach incorporated four design choices that remain influential: the Differential Sharpe Ratio as a risk-adjusted reward, a recurrent network architecture for temporal state encoding, direct policy optimization rather than a predict-then-trade pipeline, and explicit transaction cost modeling. Moody demonstrated that RL could outperform supervised learning baselines on S&P 500 futures, establishing feasibility of the approach and showing that optimizing risk-adjusted return directly was often superior to predicting intermediate outcomes.

Moody and Saffell (2001) extended this framework to multi-asset portfolios, validating that RL-optimized policies could generate positive risk-adjusted returns after transaction costs — a stringent test of practical viability.

2.7.2. From Discrete to Continuous Action Spaces

Early value-based approaches applied Q-learning with discrete buy/sell/hold actions to equity and futures markets (Neuneier 1998; J. W. Lee et al. 2007). These studies established that RL could learn profitable patterns from historical data, but also identified persistent problems: sparse rewards, delayed credit assignment, and out-of-sample performance degradation. More fundamentally, discrete action spaces force an artificial granularity on position sizing — a coarse grid loses control precision while a fine grid reintroduces the curse of dimensionality.

Gold (2003) and Dempster and Romahi (2002) extended the direct and recurrent reinforcement learning tradition to foreign-exchange trading systems. Their relevance is that the trading policy maps market state directly into positions rather than into an intermediate price forecast. The evidence is also cautionary: performance varied substantially across currency pairs and depended on the choice of model and system parameters, making early direct-RL results better interpreted as feasibility evidence than as a robust trading edge.

2.7.3. Deep Reinforcement Learning Era

The introduction of deep function approximators to RL trading extended applicability to high-dimensional state spaces.

Xiong et al. (2018) and Deng et al. (2017) adapted DQN for equity and cryptocurrency trading, demonstrating that convolutional networks could extract useful features from raw price data. However, both studies noted high variance across runs — a characteristic instability of value-based methods under noisy financial rewards.

Jiang, Xu, and Liang (2017) proposed the portfolio-vector-memory (PVM) architecture, combining policy gradients with recurrent networks for cryptocurrency portfolio management. Their fully differentiable end-to-end design incorporated current holdings into the state and penalized transaction costs explicitly, outperforming Markowitz mean-variance allocation in backtests across 12 cryptocurrencies.

An earlier cryptocurrency portfolio study by Jiang and Liang (2017) used a convolutional network to map historical prices directly into continuous portfolio weights under a deterministic policy-gradient-style objective. Its 30-minute Poloniex backtests are relevant because they treat trading as direct portfolio-action optimization, but the assumptions of immediate execution and no market impact remain far from tick-level order-book trading.

Liang et al. (2018) applied DDPG to stock trading and conducted ablation studies showing that: (i) DDPG outperformed DQN for continuous position sizing, (ii) Sharpe-based rewards improved out-of-sample generalization, and (iii) transaction cost modeling was critical for realistic performance assessment.

Majidi, Shamsi, and Marvasti (2024) provided a focused TD3 study on single-asset trading for AMZN and BTC using daily data and a continuous action space in $[-1, 1]$. Their minimal state representation — lagged close-return windows rather than large technical-indicator sets — reported improved return and Sharpe ratio for TD3 versus discrete variants and buy-and-hold, though results were mixed across assets. This study is best interpreted as evidence that continuous action sizing can be beneficial under realistic frictions, not as definitive proof of robust alpha across market regimes.

Kabbani and Duman (2022) proposed a TD3-based framework incorporating account state (cash and holdings), price features, technical indicators, and optional FinBERT-derived news

sentiment. Richer state designs improved both return and Sharpe ratio relative to a close-price-only baseline, with a final 10-stock evaluation reporting a Sharpe ratio of 2.68. This figure warrants scrutiny: annualized Sharpe ratios above 2 are rare in live trading and typically indicate either insufficient out-of-sample discipline, favorable selection among multiple configurations, or evaluation periods that coincide with a strong directional trend. The study does not report results across multiple random seeds or under realistic spread-crossing costs with a frozen policy, making it difficult to determine whether the reported performance reflects genuine signal or overfitting to the evaluation window. Methodologically the paper supports the practical value of feature enrichment in continuous-action actor-critic trading, but it also illustrates a broader pattern in the RL trading literature: without standardized evaluation protocols — multi-seed reporting, held-out test periods not used for configuration selection, and explicit cost assumptions — performance claims are difficult to assess on their own terms.

Yang et al. (2020) evaluated an ensemble stock-trading strategy built from PPO, A2C, and DDPG on Dow Jones constituents. The study is relevant as an actor-critic comparison in a liquid equity universe, but it is not evidence about millisecond-level LOB trading; its state representation, trading frequency, and execution assumptions differ from the setting studied in this thesis.

Wang et al. (2019) proposed AlphaStock, which combines a Sharpe-oriented RL objective with history-state and cross-asset attention networks. Its contribution is mainly about representation and interpretability in stock selection: the attention mechanism gives a partial account of which asset characteristics drive portfolio construction. This supports the broader claim that state representation matters materially in RL trading, but it does not address order-book-driven single-asset exposure control.

Zhang, Zohren, and Roberts (2020) provide a broader review of deep RL trading systems, emphasizing that algorithm choice cannot be separated from state representation, action design, reward definition, transaction costs, and evaluation protocol. The review is useful here as a synthesis of design dimensions rather than as evidence for the superiority of any single algorithm.

2.7.4. Comparative Evidence and Limitations

Algorithm comparisons and reproducibility. Liu et al. (2020) illustrates a later move toward reproducible DRL trading pipelines by implementing several standard algorithms, including DQN, DDPG, PPO, SAC, A2C, and TD3, within common market environments and backtesting workflows. Its relevance is infrastructural rather than evidentiary: it shows that applied trading studies increasingly require standardized environments, comparable baselines, and explicit market-friction assumptions before algorithm rankings can be interpreted.

López de Prado (2018) compared deep RL against traditional quantitative strategies (momentum, mean reversion, pairs trading), noting that RL exhibited superior adaptability during regime changes while traditional strategies offered better interpretability. Hybrid approaches combining domain knowledge with RL often outperformed either in isolation.

Overfitting and generalization. Krauss, Do, and Huck (2017) demonstrated that many published RL trading strategies fail to generalize out-of-sample, attributing this to high noise-to-signal ratios in financial data, limited training data relative to model complexity, and non-stationarity that invalidates training-period patterns. The implication is that evaluation methodology — strict temporal splits, multi-seed reporting, realistic cost assumptions — matters as much as algorithm choice.

Reward specification. Millea (2021) emphasizes that reward shaping is one of the central design choices in DRL trading. Misspecified rewards can lead to metric gaming, excessive turnover, ignoring tail risks, or misalignment with actual investor preferences. The Differential Sharpe Ratio (Moody and Saffell 1998) addresses some of these concerns by targeting risk-adjusted return directly, but does not fully resolve the tension between reward simplicity and the richness of real trading objectives.

2.7.5. Structured Comparison of Representative Studies

Study	**Market / Task**	**State Variables**	**Action Space**	**Reward / Objective**	**Main Caveat**
Moody & Saffell (1998)	Futures trading	Price-derived signals + recurrent state	Continuous position control	Differential Sharpe Ratio	Limited modern reproducibility standards

(continued on next page)

(continued from previous page)

Study	**Market / Task**	**State Variables**	**Action Space**	**Reward / Objective**	**Main Caveat**
Jiang et al. (2017)	Crypto portfolio allocation	Price tensor + previous portfolio weights	Continuous portfolio weights	Commission-aware portfolio growth	Backtest dependence on selected period and universe
Liang et al. (2018)	Stock portfolio management	Market features + macro variables	Continuous (DDPG)	Return with friction-aware setup	Sensitivity to architecture and hyperparameters
Majidi et al. (2024)	Single-asset AMZN/BTC	Close-return sliding window (autoregressive)	Continuous $[-1, 1]$	Log-return with transaction cost	Mixed performance vs. buy-and-hold across assets
Kabbani & Duman (2022)	Multi-asset stock trading	Cash + holdings + close + indicators + sentiment	Continuous (TD3)	Portfolio return / Sharpe-oriented	Sharpe 2.68 likely optimistic; no multi-seed or frozen-policy evaluation
Yang et al. (2020)	Dow Jones stock trading	Prices + technical indicators + account state	Mixed actor-critic ensemble	Portfolio value / risk-adjusted comparison	Equity portfolio setting, not tick-level LOB trading

The literature reviewed here points to several patterns relevant to the design choices in this thesis. Across the representative studies discussed above, continuous-action actor-critic methods are frequently preferred over discrete-action alternatives for position-sizing tasks (Liang et al. 2018; Majidi, Shamsi, and Marvasti 2024; Kabbani and Duman 2022). Risk-adjusted reward signals, especially Differential Sharpe Ratio formulations, also appear promising for out-of-sample generalization relative to raw return objectives, though such claims remain highly sensitive to the evaluation protocol and cost assumptions. Explicit transaction cost modeling is a prerequisite for any realistic performance claim at the frequencies considered here.

Viewed by design dimension, the strongest lessons are narrower than the headline returns

reported in many papers. State representations must include the information needed for the action being optimized; action spaces should match the economic decision, especially when the task is exposure sizing; rewards must penalize costs and avoid encouraging excessive turnover; and validation must use held-out periods, frozen policies, and repeated runs where stochastic training is material. Without these elements, a positive backtest is best interpreted as a proof of concept rather than evidence of a robust trading edge.

Within the representative literature reviewed here, a comparatively underexplored design combination is the joint use of order-book-derived state variables, continuous-control RL, and an explicit microstructure-oriented state design. Much of the applied literature relies primarily on OHLCV or other aggregated price representations, while studies using deeper limit-order-book information remain fewer and methodologically heterogeneous. The present thesis is positioned within that less represented part of the literature by constructing the agent's observation vector from 10-level order book data and by evaluating a continuous-control agent under the modeling assumptions stated in later chapters.

2.8. Implications for This Thesis

The literature reviewed in this chapter motivates the empirical design of the thesis along five dimensions. First, market microstructure theory and evidence support the use of LOB-derived state variables rather than relying only on bar-level price summaries. Spread, imbalance, order-flow, fair-value, and intraday-regime features each correspond to a distinct mechanism discussed in the microstructure literature.

Second, the reviewed evidence also limits the interpretation of these features. They are candidate state variables, not guaranteed trading signals. Their value depends on whether they provide information available before the agent acts and whether any resulting policy remains useful after transaction costs and out-of-sample evaluation.

Third, the comparison with supervised and rule-based approaches motivates direct optimization of trading decisions. Forecasting models can be informative, but a separate rule is still required to translate predictions into exposure. In this thesis, the agent instead chooses a continuous portfolio position directly, making the action space part of the optimization problem.

Fourth, the applied RL literature motivates the use of continuous-control actor-critic methods

and risk-aware reward design, while also requiring caution about reported performance. Prior studies show that RL trading results are sensitive to transaction costs, benchmark choice, random seeds, and validation protocol. For that reason, the later empirical chapters separate training reward from financial evaluation metrics and assess the learned policy on held-out data.

Finally, the contribution of the thesis follows from the combination of these design choices: a microstructure-aware state representation, continuous exposure control, transaction-cost-aware evaluation, and explicit robustness assessment within a controlled single-asset HFT simulation. The objective is not to claim a deployable trading system, but to test whether this particular design combination can produce credible out-of-sample evidence under the stated simulation assumptions.

3. Reinforcement Learning

This chapter develops the reinforcement learning foundations required for the trading agent implemented here. It is organized in three parts. The first part introduces the core components of an RL system — the formal vocabulary of states, actions, rewards, policies, and value functions that all subsequent algorithms inherit. The second part classifies RL algorithms along the dimensions that matter most for the trading problem: model-free vs. model-based, and on-policy vs. off-policy. The third part traces the algorithmic lineage from early value-based methods to TD3, showing how these properties converge on TD3 as the algorithm of choice for continuous-control trading.

3.1. Components of a Reinforcement Learning System

Reinforcement learning systems address sequential decision-making problems by selecting actions that maximize cumulative discounted future rewards. This subsection defines the core components using trading as the primary illustrative context, drawing on Sutton and Barto (2018).

Formally, the trading problem can be described as an approximate Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition kernel, $R(s, a, s')$ is the reward function, and γ is the discount factor. In financial markets, this formulation is necessarily an approximation. The agent observes engineered order-book features rather than the full market state, so the empirical problem is better understood as a partially observed decision problem represented through an approximate MDP. Relevant information may be omitted because of hidden liquidity, unobserved order-flow history, latent intraday regimes, queue priority, and other market participants' private decisions. The transition process is also affected by non-stationarity and intraday seasonality. The state representation used in this thesis should therefore be understood as an attempt to construct a useful decision state under partial observability, not as proof that the market itself is fully Markovian.

3.1.1. Environment

The environment defines the space of possible states and the rules governing transitions between them. In the trading setting studied here, the environment is the simulated order-

book market: it receives a portfolio-exposure action from the agent at each time step, updates the portfolio value according to price changes and transaction costs, and returns the new state and reward. The environment encapsulates everything outside the agent's direct control — price dynamics, spread, and execution frictions represented by the simulator — under a set of explicit simulation assumptions.

3.1.2. State

The state s_t is a snapshot of the environment at time t that contains sufficient information for the agent to select an action. In the HFT setting of the state is a vector of engineered microstructure features derived from the limit order book: imbalance measures, spread, order flow signals, and temporal encodings. States can be terminal, ending an episode, or non-terminal, continuing the interaction sequence. The Markov property requires that the optimal action depend only on s_t and not on the full history; the feature engineering in Chapter 4 is designed to approximate this property.

3.1.3. Action

The action a_t is the decision the agent takes given state s_t. In this thesis the action space is continuous: $a_t \in [-1, 1]$, representing the target portfolio exposure from maximum short to maximum long. The continuous formulation is important because it allows the agent to express varying degrees of conviction through position size, rather than being restricted to a binary buy/sell decision.

3.1.4. Policy

The policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$ maps states to actions. A deterministic policy is used — given the same state, the policy always produces the same action. This is appropriate for a trading agent expected to behave consistently under identical market conditions. During training, exploration noise is added to the policy output to encourage discovery of profitable behaviors; at evaluation time the policy is used without noise.

3.1.5. Reward

The reward r_t is a scalar signal returned by the environment after the agent takes action a_t in state s_t. It provides the only learning signal available to the agent — there are no labels,

no supervisor. The reward function is therefore a critical design choice: it determines what the agent actually optimizes. In trading, rewards are typically derived from portfolio returns, possibly adjusted for risk and transaction costs. The specific reward formulation used here is developed in Chapter 4.

3.1.6. Value Function

The value function $V^\pi(s)$ estimates the expected cumulative discounted reward starting from state s under policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right]$$

where $\gamma \in [0, 1]$ is the discount factor. The value function allows the agent to reason about long-run consequences rather than only immediate rewards — a necessary capability in trading, where the cost of a position change today affects future returns through transaction costs and portfolio path dependence. The action-value function $Q^\pi(s, a)$ extends this to state-action pairs and forms the basis of the critic networks in actor-critic algorithms.

3.2. Classifying RL Algorithms

3.2.1. Model-free vs Model-based

Reinforcement learning algorithms divide into two broad families depending on whether they require a model of the environment.

Model-based methods learn or are given a transition model $P(s'|s, a)$ and use it to plan ahead — computing value functions by rolling out simulated trajectories rather than purely from observed data. Dyna-Q and AlphaZero are canonical examples. When the model is accurate, planning is data-efficient; when the model is wrong, planning amplifies errors. In financial markets, transition dynamics are unknown and non-stationary, making a reliable model practically unattainable.

Model-free methods learn directly from sampled transitions without building a transition model. They are less sample-efficient, and they still rely on the sampled experience being informative for the target environment and on the function approximator generalizing usefully across states and actions. TD3 belongs to this family.

3.2.2. Value-Based, Policy-Based, and Actor-Critic

A second dimension classifies algorithms by what they learn and optimize.

Value-based methods learn a value function and derive a policy implicitly by acting greedily with respect to it. Q-learning and its deep variant DQN (Mnih et al. (2015)) are prototypical examples. They work well in discrete action spaces but struggle when actions are continuous, because the greedy step $\arg\max_a Q(s, a)$ becomes intractable over an uncountable set.

Policy-based methods directly parameterize and optimize the policy π_θ. REINFORCE (Williams (1992)) updates parameters by gradient ascent on expected return:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}\left[\sum_t r_t\right]$$

Policy gradients handle continuous actions naturally but suffer from high variance, requiring many samples to obtain a reliable gradient signal.

Actor-critic methods combine both: an actor μ_θ selects actions and a critic $Q_w(s, a)$ evaluates them, using the critic's lower-variance estimates to guide actor updates. The full actor-critic derivation and TD3's specific extensions are given in Section 3.3.

3.2.3. On-Policy vs Off-Policy

On-policy methods evaluate and improve the same policy used to collect experience. SARSA is a canonical example, updating with:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

where a_{t+1} is drawn from the current policy. On-policy methods tend to be stable but wasteful: each sample is used once and then discarded.

Off-policy methods decouple the data-collection policy (the behavior policy) from the policy being improved (the target policy). Q-learning updates with the greedy maximum:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

regardless of how a_{t+1} was actually selected. This separation allows experience from a replay buffer — collected under earlier or exploratory policies — to be reused many times. In a financial simulator, however, replay does not create counterfactual observations of how the market would have evolved under actions not actually taken. It is valid only under the maintained simulation assumption that the agent is price-taking and that its exposure choice affects portfolio wealth but not the subsequent order-book path. TD3 is off-policy and uses a replay buffer to reuse collected transitions under this simulator design; the implications for sample efficiency and execution assumptions in financial settings are discussed in Section 3.3.

3.3. Actor-Critic Methods for Continuous Control

The development of reinforcement learning algorithms is a series of conceptual breaks , each motivated by a limitation in the preceding framework. This section traces the specific lineage that leads to the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm, which is employed here. The survey is not exhaustive; it focuses on the algorithmic properties that are directly relevant to the continuous-action, single-asset trading problem formulated in Chapter 4.

3.3.1. Value-Based Methods and Their Limitations in Continuous Control

The foundational algorithms of the field — Q-learning (Watkins and Dayan 1992) and its on-policy variant SARSA (Rummery and Niranjan 1994) — operate over tabular representations of state-action value functions. Both methods converge to the optimal policy under mild conditions, but only when the state-action space is small enough to enumerate explicitly. In a trading context with continuous price-derived features, the state space is effectively infinite, and a tabular representation is computationally infeasible.

Deep Q-Networks (DQN) (Mnih et al. 2015) resolved the scalability problem by replacing the lookup table with a deep neural network, enabling generalization across states. Two architectural innovations were critical: experience replay, which decorrelates training samples by storing and randomly resampling past transitions, and a periodically frozen target network, which stabilizes the Bellman update by preventing the target from shifting at every gradient step. DQN achieved human-level control across a wide range of tasks, but it inherits a structural constraint from Q-learning: it requires a discrete action space, because the max operator in the Bellman target presupposes that all actions can be enumerated. For a trading

agent that outputs a continuous portfolio weight in $[-1, 1]$, this is not an acceptable limitation. Discretizing the action space into a finite grid resolves the representation issue formally, but it reintroduces the granularity problem: a coarse grid loses the benefits of continuous control, while a fine grid makes the maximization step computationally expensive and recovers the curse of dimensionality in higher-dimensional portfolio settings.

3.3.2. Policy Gradient Methods and the Actor-Critic Architecture

A distinct class of algorithms avoids the maximization problem by parameterizing the policy directly and optimizing it through gradient ascent on expected cumulative reward. The foundational result, REINFORCE (Williams 1992), uses Monte Carlo estimates of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right]$$

where G_t is the return from time t. While REINFORCE applies naturally to continuous action spaces through a parameterized distribution (for example, a Gaussian policy), it suffers from high variance in the gradient estimate because G_t is computed from a full episode trajectory, making it sensitive to trajectory length and reward scale.

The actor-critic architecture reduces this variance by replacing G_t with a learned advantage estimate $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, maintained by a separate critic network. The actor updates the policy in the direction of higher-valued actions, while the critic provides a lower-variance baseline. This separation is the architectural foundation of all subsequent continuous-action algorithms, including TD3.

3.3.3. DDPG: Deterministic Policies for Continuous Control

Lillicrap et al. (2016) introduced Deep Deterministic Policy Gradient (DDPG), which extends the actor-critic framework to deterministic policies over continuous action spaces. Rather than sampling an action from a distribution, the actor outputs a single action $\mu_\theta(s)$ directly. The policy gradient simplifies to:

$$\nabla_\theta J(\theta) = \mathbb{E}_s \left[\nabla_a Q_\phi(s, a)\big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s) \right]$$

DDPG inherits the experience replay and target network mechanisms from DQN, making it an off-policy algorithm and therefore sample-efficient. However, empirical studies consistently report two failure modes in DDPG. First, the critic tends to overestimate action values because actor optimization can exploit positive approximation errors in the critic, and bootstrapped targets can propagate those overestimated values through subsequent updates. Overestimated Q-values then push the policy toward actions that exploit the critic's errors rather than the true reward signal. Second, the algorithm is sensitive to the relative frequency of actor and critic updates: updating the actor before the critic has converged amplifies the overestimation problem. In financial environments, where rewards are noisy and non-stationary, these instabilities are particularly damaging because the critic cannot distinguish between genuine market signal and approximation artifacts.

3.3.4. PPO: On-Policy Actor-Critic with Clipped Objectives

Schulman et al. (2017) introduced Proximal Policy Optimization (PPO), which balances the sample efficiency of off-policy methods with the stability of on-policy updates. PPO optimizes a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio between the new and old policies, ϵ is a clipping hyperparameter (typically 0.2), and $\hat{A}_t$ is the advantage estimate.

The clipping mechanism prevents excessively large policy updates that would collapse performance, making PPO more robust to hyperparameter choices than earlier policy gradient methods such as Vanilla Policy Gradient (Sutton et al. 2000) and Trust Region Policy Optimization (TRPO) (Schulman et al. 2015). Unlike TD3 and DDPG, PPO is on-policy: it optimizes against the most recent policy and does not use a replay buffer. This trades sample efficiency for stability, which can be advantageous in environments where the value function is difficult to approximate.

PPO also maintains a stochastic policy rather than the deterministic policies used by DDPG and TD3. At each state, the policy outputs a distribution over actions (typically Gaussian), and actions are sampled from this distribution. Exploration is handled naturally through the entropy of the action distribution, which can be explicitly encouraged through an entropy

bonus term in the objective:

$$L^{CLIP+ENT}(\theta) = L^{CLIP}(\theta) - c_1 \mathcal{H}(\pi_\theta)$$

where $\mathcal{H}(\pi_\theta)$ is the policy entropy and c_1 is a coefficient controlling the strength of the exploration bonus.

For trading applications, PPO's stochastic policy provides built-in exploration without requiring explicit noise injection during training, and its clipped objective helps avoid catastrophic performance drops that can occur in high-variance financial environments. However, its on-policy nature means each transition is used only once before being discarded, which is computationally expensive when training on limited historical data. This is a particularly relevant consideration for high-frequency trading, where the available order-book sample may be limited relative to the complexity of the value function approximation.

3.3.5. TD3: Addressing the Instability of Deterministic Actor-Critic Methods

Fujimoto, Hoof, and Meger (2018) introduced TD3 as a direct response to DDPG's failure modes, proposing three targeted mechanisms.

Clipped double Q-learning. Two independent critic networks Q_{ϕ_1} and Q_{ϕ_2} are maintained, and the Bellman target is constructed using the minimum of their estimates:

$$y = r + \gamma \min_{i=1,2} Q_{\phi'_i}(s', \tilde{a}')$$

Taking the minimum provides a conservative estimate of target value and substantially reduces overestimation bias. This is analogous to the double Q-learning correction (Hasselt, Guez, and Silver 2016), adapted to the deterministic actor- critic setting.

Delayed policy updates. The actor and target networks are updated less frequently than the critics — typically once for every two critic updates. This allows the value estimates to stabilize before they are used to compute the policy gradient, breaking the feedback loop between an unstable critic and an erroneous actor update.

Target policy smoothing. When computing Bellman targets, noise is added to the target action $\tilde{a}' = \mu_{\theta'}(s') + \epsilon$, with ϵ clipped to a small interval. This prevents the critics from

fitting sharp value peaks caused by function approximation, acting as a regularizer on the value landscape.

Together, these three modifications produce an algorithm that is substantially more reliable than DDPG on standard continuous control benchmarks while retaining its off-policy sample efficiency. This matters for the present thesis for a specific reason: the agent is trained in a price-taking historical-market simulator where each transition can be replayed because the agent's action changes portfolio wealth but not the subsequent order-book path. Replay therefore improves computational use of the simulated experience, but only under this execution assumption; it should not be interpreted as evidence that the agent has observed all counterfactual market reactions to alternative trades.

TD3 is selected as the main algorithm because it matches four requirements of the empirical design. First, the action is a continuous target exposure in $[-1, 1]$, so discretized value-based methods such as DQN would require an arbitrary action grid and would make position sizing a modelling artifact. Second, the evaluation policy is deterministic, so a deterministic actor provides a direct mapping from state features to exposure without adding an entropy term to the objective. Third, compared with DDPG, TD3 directly addresses critic overestimation and unstable actor updates, both of which are acute when rewards are noisy and weakly informative. Fourth, compared with on-policy methods such as PPO, TD3 can reuse simulator transitions through replay, which is important when the available high-frequency sample is limited and live data collection is outside the scope of the thesis.

This selection does not imply that TD3 dominates all alternatives. SAC (Haarnoja et al. 2018) is a strong continuous-control alternative because its stochastic policy and maximum-entropy objective can improve exploration, but it optimizes an entropy-regularized criterion rather than the deterministic exposure policy evaluated here. PPO is often robust in practice, but its on-policy updates are less aligned with replay-based learning from a fixed historical simulation. DDPG is the closest baseline conceptually, but TD3 was introduced precisely to reduce DDPG's overestimation and policy-update instability. TD3 is therefore not chosen as a generally superior trading method; it is chosen as a defensible match to the continuous-action, deterministic-policy, replay-based simulation design used in this thesis.

The choice of TD3 should be read together with the trading constraints discussed in Chapter 2. Those sections motivate continuous exposure control, off-policy learning from historical

data, and caution about transaction costs, non-stationarity, and partial observability. TD3 matches the first two requirements through deterministic continuous actions and replay-based learning. It does not solve the remaining market-design problems. For that reason, the empirical chapters must still evaluate performance under explicit transaction costs, chronological validation, and the simulation assumptions stated in later sections.

3.3.6. Algorithm Comparison: TD3, PPO, and DDPG for Trading

The three algorithms considered here differ along three dimensions that are particularly relevant to the continuous-action trading problem: learning paradigm, policy type, and stability mechanisms. These differences determine how to interpret empirical comparisons and why TD3 is selected over the alternatives.

On-policy vs. off-policy learning. TD3 and DDPG are off-policy algorithms that use experience replay, allowing each transition to be reused many times across multiple gradient updates. This is valuable when training data is limited, as is typical with high-frequency historical order-book data. Replay buffers also decorrelate consecutive transitions, reducing the variance of gradient estimates. PPO is on-policy and uses each transition only once before discarding it, requiring more training iterations for comparable sample efficiency. However, on-policy learning can be more stable when the environment distribution shifts rapidly, as financial markets do.

Deterministic vs. stochastic policies. TD3 and DDPG output deterministic actions $a = \mu_\theta(s)$, matching the evaluation objective of a single exposure decision per state. During training, exploration is handled by adding noise to the deterministic action: $a = \mu_\theta(s) + \epsilon$, with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. At evaluation time, the noise is removed, yielding a consistent, interpretable policy. PPO maintains a stochastic policy that outputs a distribution over actions (typically Gaussian), with actions sampled from this distribution. Exploration is handled naturally through the entropy of the action distribution rather than explicit noise injection. Deterministic policies are preferable when the final deployed system must be consistent and interpretable, while stochastic policies can provide more flexible exploration during training.

Stability mechanisms. Each algorithm addresses stability challenges through different mechanisms. DDPG suffers from Q-value overestimation because the actor can exploit positive approximation errors in the critic, and these overestimates propagate through boot-

strapped Bellman updates. TD3 addresses this through clipped double Q-learning (taking the minimum of two critic estimates) and delayed policy updates (updating the actor less frequently than the critics). PPO prevents large policy changes through its clipped surrogate objective, which limits how much the policy can change in a single update. Both TD3 and PPO are substantially more stable than vanilla DDPG, but through different mechanisms: TD3 targets value estimation errors, while PPO constrains policy changes directly.

Sample efficiency vs. stability trade-off. The practical implication of these differences is a trade-off between sample efficiency and training stability. TD3 and DDPG can make efficient use of limited training data through replay, but DDPG's instability can make it difficult to train reliably. PPO is typically more stable but requires more data to achieve comparable performance due to its on-policy nature. For high-frequency trading applications, where training data may be limited but stability is critical for avoiding catastrophic losses, TD3's combination of off-policy efficiency and stability mechanisms makes it a strong candidate.

Table: Algorithm Properties for Trading Applications

Property	DDPG	TD3	PPO
Policy Type	Deterministic	Deterministic	Stochastic
Learning Paradigm	Off-policy (replay)	Off-policy (replay)	On-policy
Exploration Mechanism	Action noise	Action noise + smoothing	Entropy bonus
Stability Mechanism	Prone to overestimation	Double Q + delayed updates	Clipped objective
Sample Efficiency	High	High	Moderate
Training Stability	Low	High	High
Hyperparameter Sensitivity	High	Moderate	Low
Suitability for Limited Data	High	High	Lower
Interpretability	High (deterministic)	High (deterministic)	Lower (stochastic)

This comparison framework guides the experimental design in Chapter 6, where TD3, PPO, and DDPG are evaluated on identical trading tasks to empirically assess how these algorith-

mic differences translate to trading performance. The results help determine whether TD3's theoretical advantages (reduced overestimation, deterministic policy) translate to practical benefits in the noisy, non-stationary trading environment, and whether PPO's on-policy stability can compensate for its lower sample efficiency.

4. Design of the Trading Agent

This chapter applies the reinforcement learning framework introduced in Chapter 3 to single-asset high-frequency trading. Each section addresses one component of the MDP formulation: the action space, state representation, reward function, value function, policy, discount factor, and step size. The choices made here are motivated by the structure and constraints of the trading problem.

4.1. Action Space

The action space defines the set of permissible decisions available to the trading agent at each time step. This design choice fundamentally shapes the agent's behavior and determines the granularity of control over portfolio positions.

4.1.1. Continuous vs Discrete Actions

Trading systems can be formulated with either discrete or continuous action spaces:

Discrete action spaces restrict the agent to a finite set of categorical decisions. The two most common formulations are the three-action set $\{-1, 0, 1\}$ representing short, neutral, and long positions, and the two-action set $\{0, 1\}$ representing only short and long positions without a neutral option.

Continuous action spaces allow the agent to output any value within a specified range, enabling smooth portfolio allocation decisions.

4.1.2. Implementation: Box Portfolio Action Space

In this implementation, the action space is defined as a continuous interval:

$$\mathcal{A} = [-1.0, 1.0]$$

where the action $a_t \in \mathcal{A}$ at time t represents the target portfolio allocation for the traded asset. The semantic interpretation is:

- $a_t = 1.0$: Maximum long exposure (fully invested long position)
- $a_t = 0.0$: Neutral position (no exposure, cash equivalent)
- $a_t = -1.0$: Maximum short exposure (fully invested short position)

- $a_t \in (0, 1)$: Partial long position scaled by magnitude
- $a_t \in (-1, 0)$: Partial short position scaled by magnitude

This formulation supports short-selling and is a standard simplification in single-asset RL trading environments applied to equity data. The simulation treats long and short positions as cost-symmetric: the transaction cost term $c \cdot |a_t - a_{t-1}|$ applies identically regardless of direction. In practice, short positions in AAPL incur securities-lending borrow costs — typically 25–75 basis points per year on the notional short — that are not captured here. For strictly intra-session episodes, the per-tick borrow cost is negligible: at 50 bps annualized and a 10-second average hold, the cost is on the order of 10^{-7} of notional. The cost-symmetry assumption is therefore inconsequential provided episodes do not span overnight. Where the agent holds short positions across sessions, reported returns represent an upper bound on achievable short-side performance. The continuous nature enables the agent to express varying degrees of conviction—for example, $a_t = 0.3$ indicates a moderately bullish stance, while $a_t = -0.8$ signals strong bearish conviction.

4.1.3. Action Constraints and Transaction Costs

The action space is bounded to prevent unrealistic leverage. In practice, unbounded actions could lead to positions exceeding available capital or margin requirements. The $[-1, 1]$ range implicitly assumes unit leverage without margin multipliers.

Transaction costs are applied proportionally to position changes. If the agent transitions from a_{t-1} to a_t, the turnover magnitude is $|a_t - a_{t-1}|$, and fees are calculated as:

$$\text{fee}_t = \text{trading_fees} \times |a_t - a_{t-1}| \times \text{notional value}$$

This incentivizes the agent to minimize unnecessary portfolio rebalancing, as frequent adjustments erode returns. During training, the RL algorithm must learn to balance the potential gains from reallocation against the friction costs incurred.

4.1.4. Action Smoothing and Exploration Noise

During training, exploration is introduced through additive Gaussian noise:

$$a_t^{\text{noisy}} = \text{clip}(a_t + \epsilon, -1.0, 1.0)$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ and σ is the exploration noise standard deviation.[3] The clipping operation ensures actions remain within valid bounds. This exploration mechanism encourages the agent to discover diverse trading strategies during early training phases while gradually converging to deterministic exploitation as learning progresses.

Additionally, TD3 employs target policy smoothing during critic updates, adding noise to target actions when computing Bellman targets. This regularization technique prevents the critics from overfitting to narrow peaks in the value function caused by function approximation errors.

4.2. State Space

The state space defines the informational representation available to the agent at each decision point. At each time step t, the agent receives an observation vector $s_t \in \mathcal{S}$ constructed from the current state of the limit order book (LOB). The design of this observation is the primary engineering decision in the empirical pipeline: the agent can only exploit structure that is present in its input.

4.2.1. Design Requirements

An effective state representation for high-frequency trading must satisfy several criteria simultaneously.

Approximation of the Markov property. The TD3 algorithm assumes that s_t contains sufficient information such that the optimal action depends only on the current state, not on the full history. Raw price levels violate this assumption because their unconditional distribution is non-stationary; a price of 180 means something very different depending on recent trajectory. Engineered features — returns, imbalances, normalized spreads — are designed to be more stationary and to encode the local market state rather than the absolute price level.

[3] The main experiment uses $\sigma = 0.1$. The range 0.1–0.3 reflects common practice in continuous-action TD3 applications; values above 0.3 tend to destabilize learning in low-signal environments such as tick-frequency returns.

Informativeness about short-term price direction. For the reward signal to be learnable, the observation must contain features that are predictive of near-term mid-price changes. The selection is therefore grounded in market microstructure theory, which identifies specific LOB quantities — imbalances, order flow, spread — as the empirically dominant predictors of short-term price movement.

Dimensionality discipline. Each additional observation dimension enlarges the input space that the policy and critic networks must generalize over. Features are included only when they carry a distinct theoretical mechanism not already captured by other features in the set.

Scope and timescale consistency. The observation is restricted to LOB-derived features. Bar-based signals such as OHLCV aggregates or technical indicators require a fixed aggregation window. This imposes a coarser timescale than the tick-level decisions being made.

For example, a one-second bar collapses dozens of individual order-book events into a single price summary. This discards the queue dynamics, imbalance shifts, and spread fluctuations that constitute the decision-relevant information at this resolution.

Cross-asset signals such as VIX or index returns face a related problem. They update at frequencies far below the inter-event times of the AAPL tick stream. This contributes no new information between consecutive book updates.

Aligning multiple tick streams at sub-second precision also introduces synchronization risk. A VIX observation and an AAPL LOB snapshot sharing a timestamp to the second are not the same market moment.

Beyond these practical constraints, restricting the state to single-asset LOB features is a deliberate experimental choice. The thesis tests whether microstructure-aware features alone are sufficient for a viable agent. Adding cross-asset signals would answer a different question and prevent clean attribution of any observed performance to LOB structure.

A natural extension for future work would be to augment the LOB observation with synchronized regime indicators. These could include intraday VIX futures at the minute level, sector ETF order flow, or options-derived skew.

This would be done once the baseline LOB-only performance is established. It would allow a controlled ablation of which additional signal sources materially improve out-of-sample results.

4.2.2. Feature Groups and Selection Rationale

The state vector is composed of five groups of features, each targeting a distinct aspect of LOB microstructure.

4.2.2.1. Imbalance Features

Volume imbalance at a given price level i is defined as:

$$\text{BookPressure}_i = \frac{V_i^{bid} - V_i^{ask}}{V_i^{bid} + V_i^{ask}}$$

Imbalance signals the relative weight of resting liquidity on each side and has been shown to predict short-term price direction (Cao, Chen, and Griffin 2004; Biais, Hillion, and Spatt 1995). The multi-level aggregate version averages this signal across the top N levels:

$$\text{OBI}_N = \frac{\sum_{i=0}^{N-1}(V_i^{bid} - V_i^{ask})}{\sum_{i=0}^{N-1}(V_i^{bid} + V_i^{ask})}$$

The multi-level formulation is more robust to noise at the best quote and captures the depth structure of the book. Both single-level (levels 0–2) and three-level aggregate variants are included.

Order count imbalance complements the volume-based measures by counting the number of resting orders rather than their aggregate size, making it sensitive to fragmentation that a single large institutional order would mask:

$$\text{OCI}_i = \frac{C_i^{bid} - C_i^{ask}}{C_i^{bid} + C_i^{ask}}$$

4.2.2.2. Fair Value Features

The microprice (Stoikov 2018) is a volume-weighted estimate of where the next transaction price is likely to occur:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$$

When the ask side is thin relative to the bid side, the microprice is pulled toward the ask

— the side more likely to move first. The microprice divergence from mid-price isolates the imbalance component of fair value as a stationary signal:

$$\text{MicroDiv} = \text{Microprice} - \frac{P_0^{bid} + P_0^{ask}}{2}$$

The volume-weighted mid-price skew extends this concept across three levels, measuring whether deep liquidity is positioned asymmetrically relative to the current mid-price.

4.2.2.3. Spread and Cost Features

The bid-ask spread, expressed in basis points, is the primary observable measure of adverse selection cost (Kyle 1985; Glosten and Milgrom 1985):

$$\text{SpreadBps} = \frac{P_0^{ask} - P_0^{bid}}{\frac{P_0^{bid}+P_0^{ask}}{2}} \times 10{,}000$$

A widening spread signals elevated information asymmetry or inventory risk among market makers. The spread ratio — current bid-ask relative to its recent rolling mean — separates the level effect from the regime signal, flagging periods of abnormally elevated adverse selection risk (Easley, López de Prado, and O'Hara 2012).

Bid and ask slopes measure the price elasticity of each side of the book across five levels (Bouchaud, Mézard, and Potters 2002), capturing market impact per unit of trading volume and serving as a proxy for Kyle's lambda (Kyle 1985):

$$\text{BidSlope} = \frac{P_0^{bid} - P_4^{bid}}{\sum_{i=0}^{4} V_i^{bid}}, \qquad \text{AskSlope} = \frac{P_4^{ask} - P_0^{ask}}{\sum_{i=0}^{4} V_i^{ask}}$$

4.2.2.4. Order Flow Features

Order flow imbalance (OFI) (Cont, Kukanov, and Stoikov 2014) measures the net directional pressure from LOB events between consecutive snapshots, conditioning on whether each event represents genuine queue growth rather than a price improvement:

$$\text{OFI}_t = \Delta V_t^{bid} \cdot \mathbf{1}\left[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}\right] - \Delta V_t^{ask} \cdot \mathbf{1}\left[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}\right]$$

Cont, Kukanov, and Stoikov (2014) establish OFI as the single strongest predictor of short-term price changes in LOB data. A rolling sum over 50 events captures persistent directional pressure over multiple tick horizons.

Signed trade flow (cumulative delta) (C. M. C. Lee and Ready 1991) is constructed directly from the trade tape rather than from book changes, providing a complementary signal that measures executed volume rather than queue dynamics:

$$\text{CumDelta}_{W,t} = \sum_{\tau=t-W+1}^{t} \text{size}_\tau \cdot \mathbf{1}[\text{side}_\tau = B] - \text{size}_\tau \cdot \mathbf{1}[\text{side}_\tau = A]$$

Queue depletion measures how rapidly the best-bid or best-ask queue is being consumed across consecutive snapshots, identifying situations where the top-of-book queue is close to exhaustion and a price move is imminent.

Odd-lot trade ratio and odd-lot imbalance (Boehmer et al. 2021) decompose trade flow by order size. In the AAPL dataset used here, 81% of trades are odd-lot sized (below 100 shares).[4] This primarily reflects retail and algorithmic small-lot activity.

A sudden drop in the odd-lot ratio signals increasing institutional participation. This indicates consequently higher adverse selection risk.

4.2.2.5. Regime and Temporal Features

Two features condition the agent on the current market regime rather than providing directional signals.

Log inter-event time (Engle 2000) captures the activity rate of the market:

$$\text{LogGap}_t = \log\big(1 + (t_{\text{ns}} - t_{\text{ns},t-1})\big)$$

Dense update bursts indicate informed order flow or a large order working through the book; sparse updates signal low activity and reduced adverse selection risk.

Mid-price acceleration is the second finite difference of the mid-price:

[4] This figure is computed from the February 25–27, 2026 AAPL MBP-10 dataset used in the main experiment and should not be interpreted as a long-run population statistic. The share of odd-lot trades in AAPL has increased substantially since the SEC's 2023 amendments to odd-lot reporting rules and may vary across periods.

$$\text{Accel}_t = M_t - 2M_{t-1} + M_{t-2}, \qquad M_t = \frac{P_0^{bid} + P_0^{ask}}{2}$$

It distinguishes momentum regimes (positive, accelerating) from mean-reversion regimes (negative, decelerating) at tick frequency.

The sine and cosine encodings of the hour-of-day capture the well-documented U-shaped intraday pattern in volume and spread (Wood, McInish, and Ord 1985):

$$\text{HourSin}_t = \sin\left(\frac{2\pi \cdot h_t}{24}\right), \qquad \text{HourCos}_t = \cos\left(\frac{2\pi \cdot h_t}{24}\right)$$

The cyclical encoding ensures temporal continuity — 09:30 and 16:00 remain structurally distinct without imposing an arbitrary ordinal distance.

4.2.2.6. Position Feature

The observation vector is optionally extended with the agent's own current portfolio exposure $a_{t-1} \in [-1, 1]$. Unlike the LOB-derived features, this quantity is not computed from market data but read from the simulated broker state at each step — it is a runtime feature appended after the static feature vector is constructed.

Including the current position makes the state partially Markovian with respect to inventory. Without it, the agent cannot distinguish between arriving at exposure 0 from a long position versus coming from a short position.

This distinction matters because the cost of the next action differs in both cases through the position-change fee. With the position feature, the agent can condition its action on whether a change would incur a fee. It can also learn to penalize unnecessary turnover. The feature is not z-score normalized. Because the position is bounded in $[-1, 1]$ by the action space and is drawn from a bimodal distribution that concentrates near the extremes (fully long or fully short), standardization would distort this natural structure: a z-score transform on a bimodal distribution produces a U-shaped output with negative variance contributions between the modes, which is worse than the raw signal for a neural-network input.[5]

[5]This is a practical consideration rather than a theoretical constraint. If the position distribution were approximately Gaussian, z-score normalization would reduce the scale mismatch with the other features. However, TD3 policies with continuous action spaces in $[-1, 1]$ tend to produce positions that cluster near the boundaries, making the raw representation both simpler and better matched to the actual distribution.

4.2.3. Observation Space and Normalization

The complete observation vector $s_t \in \mathbb{R}^d$ is formed by concatenating all engineered features. The exact dimensionality d is scenario-dependent and specified in the configuration files governing each experiment.

All features except cyclical time encodings and the position feature are standardized with causal running statistics:

$$\tilde{x}_{i,t} = \frac{x_{i,t} - \mu_{i,t}}{\sigma_{i,t} + \epsilon}$$

The mean $\mu_{i,t}$ and standard deviation $\sigma_{i,t}$ are updated online from observations available up to timestep t within the current chronological split and trading session. The running statistics are reset at detected session boundaries, preventing previous-day market conditions from contaminating the opening representation of the next session. Cyclical encodings are inherently bounded in $[-1, 1]$ and require no normalization. The position feature is likewise not standardized; as discussed in the preceding section, it is already bounded in $[-1, 1]$ and its bimodal distribution makes z-score normalization counterproductive.

The formal observation space is:

$$\mathcal{S} = \mathbb{R}^d$$

In practice, after standardization, feature values concentrate in $[-3, 3]$ under approximate Gaussian assumptions. The policy and value networks must generalize to out-of-distribution observations that arise during stress episodes or regime shifts not present in the training sample.

4.2.4. On the Exclusion of Raw LOB Columns

Raw price and size columns — that is, the unprocessed bid and ask prices and sizes at each level — were deliberately excluded from the observation vector. Prior work in the deep learning tradition feeds the raw LOB matrix directly to the network (Zhang, Zohren, and Roberts 2019), allowing convolutional or recurrent architectures to discover structure from data without hand-engineered intermediaries. This is a coherent approach when the model

itself is the feature extractor.

In the present setup, however, comprehensive microstructure features with established theoretical mechanisms are the primary input to the agent. Including raw columns alongside them introduces redundancy without adding an interpretable signal: normalized bid sizes at each level carry substantially the same information as the imbalance features derived from them, and normalized absolute prices at tick frequency are near-meaningless once the local market state is already encoded through microprice divergence, OFI, and spread measures. Mixing both paradigms would imply that the engineered features are insufficient, which is inconsistent with the selection rationale described above. The observation vector is therefore composed entirely of theoretically motivated derived features.

4.2.5. Feature Selection Process

Feature selection follows a two-stage procedure that combines theoretical constraints with empirical ranking.

Stage 1: Theory-constrained candidate pool. The LOB feature registry contains over forty computable statistics. The candidate pool is restricted to features that represent a distinct mechanism identified in the market microstructure literature. Each of the five groups — imbalance, fair value, spread, order flow, and regime — corresponds to a specific theoretical construct: queue pressure (Cao, Chen, and Griffin 2004; Biais, Hillion, and Spatt 1995), fair value estimation (Stoikov 2018), adverse selection cost (Kyle 1985; Glosten and Milgrom 1985), directional flow (Cont, Kukanov, and Stoikov 2014; C. M. C. Lee and Ready 1991), and intraday regime context (Engle 2000; Wood, McInish, and Ord 1985). Features not grounded in this literature — raw price and size columns, or purely data-driven aggregates without a microstructure interpretation — are excluded from consideration regardless of their statistical properties. This stage prevents data-mining by ensuring that every candidate has a prior economic justification before any data is examined.

Stage 2: IC/ICIR ranking within the theory-constrained pool. Within each group, multiple implementations of the same theoretical concept exist: OFI at different rolling horizons, spread ratios at different windows, imbalance at different depth levels. Theory cannot resolve which implementations are informative or which combinations are redundant; this is an empirical question. Each candidate is scored by its Information Coefficient Information

Ratio (ICIR) — the mean IC divided by the standard deviation of IC over rolling windows of the validation split, where IC is the Spearman rank correlation between the feature and the forward log-return proxy. Redundancy within the selected set is controlled via a conditional IC filter: after each feature is selected, its linear signal is regressed out of remaining candidates, and ICIR is recomputed on the residuals. A candidate is included only if it retains incremental predictive content beyond what the already-selected features collectively capture. The full procedure is described in Appendix F.

The result is a state space whose composition is jointly justified: every feature is grounded in theory (Stage 1) and empirically non-redundant within the theory-constrained pool (Stage 2). The two stages address different questions. Theory answers: which mechanisms belong in the state space? IC/ICIR answers: among the available implementations of those mechanisms, which combinations contribute distinct signal?

The selection is operationalized through a registry-driven configuration pipeline. Feature names defined in the scenario YAML file resolve to computation functions in the feature registry at runtime; adding, removing, or reordering features requires only a change to the configuration file, not to source code. The active feature set for the main experiment is defined in `src/configs/features/hft_lob_features_causal.yaml` and referenced by the scenario file `src/configs/scenarios/aapl/td3_hft_lob_state_space.yaml`.

The final state vector has dimensionality $d = 26$: 25 LOB-derived features across five groups (5 imbalance, 3 fair value, 6 spread and cost, 7 order flow, 4 regime and temporal) plus the current portfolio exposure appended at runtime. All features except the cyclical hour encodings are normalized using causal session-aware running statistics as described in the preceding section. The complete feature specification with parameter values is listed in Appendix A.

4.3. Reward Function

The reward function determines what the agent actually optimizes. The thesis implements direct optimization of trading performance: the agent observes the market state, selects a continuous portfolio exposure $a_t \in [-1, 1]$, and receives a scalar reward reflecting realized return net of transaction costs. A baseline formulation of this reward is:

$$r_t = a_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} - c \cdot |a_t - a_{t-1}|$$

where a_{t-1} is the exposure held during the period, P_t is price at the end of the period, and $c \cdot |a_t - a_{t-1}|$ is the transaction cost proportional to the size of the position change. The agent learns a deterministic policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$ to maximize expected cumulative reward:

$$\max_\theta \mathbb{E}\left[\sum_{t=1}^{T} \gamma^{t-1} r_t\right]$$

In trading, two properties are essential for the reward to work well in practice: the signal should be risk-adjusted so the agent does not maximize returns by taking excessive volatility, and it should be computable recursively so that no look-ahead into future returns is required at each training step. The following sections explain why the baseline formulation above requires refinement in a high-frequency setting, and how the Differential Sharpe Ratio addresses both requirements.

4.3.1. Why Raw Log-Return Is Insufficient

The simplest reward is the single-step log-return:

$$r_t = \log \frac{P_t}{P_{t-1}} \cdot a_{t-1}$$

At tick frequency, this signal is dominated by microstructure noise. Individual price changes on an MBP-10 order book are a fraction of the spread. The agent cannot distinguish genuine directional movement from quote flickering in a single step.

Training on raw log-return produces highly variable gradient estimates. It also tends to reward high-turnover behavior regardless of risk.

A related alternative is the Sharpe ratio computed over a fixed rolling window. This approach requires storing the full window of past returns at each step. It also introduces a sharp discontinuity when old returns drop out of it.[6]

[6]When the oldest return in a fixed window is unusually large or small, its removal produces an abrupt shift in the estimated mean and variance — and therefore in the Sharpe ratio — that is unrelated to the current action. The agent cannot distinguish this accounting artifact from a genuine change in performance, introducing spurious gradient variance. Exponential weighting in the DSR avoids this because old returns

Neither property is desirable in an online RL setting.

4.3.2. Differential Sharpe Ratio

The Differential Sharpe Ratio (Moody and Saffell 1998) resolves both issues. It is defined as the first-order approximation of how the current return R_t changes the exponentially weighted Sharpe ratio:

$$D_t = \frac{B_{t-1}\,\Delta A_t - \frac{1}{2} A_{t-1}\,\Delta B_t}{\left(B_{t-1} - A_{t-1}^2\right)^{3/2}}$$

where A_t and B_t are exponentially weighted estimates of the first and second moments of returns:

$$A_t = \eta R_t + (1-\eta)\,A_{t-1}, \qquad B_t = \eta R_t^2 + (1-\eta)\,B_{t-1}$$

and $\Delta A_t = A_t - A_{t-1}$, $\Delta B_t = B_t - B_{t-1}$. At $t = 0$, $A_0 = B_0 = 0$.

The smoothing parameter η controls the effective memory of the estimator: larger η makes the estimate more reactive to recent returns, smaller η averages over a longer history. The effective window length is approximately $1/\eta$ steps. In the empirical experiments, $\eta = 0.01$, giving an effective memory of roughly 100 ticks — long enough to smooth microstructure noise while short enough to adapt to intraday regime shifts.

The DSR has two properties that make it well-suited to this setting. First, it penalizes return volatility intrinsically. A sequence of returns with the same mean but higher variance produces a lower DSR. This discourages the agent from achieving returns through high-variance position flipping.

Second, the DSR is recursively computable from A_{t-1} and B_{t-1} alone. This means no fixed window, no look-ahead, and no discontinuity as old observations expire.

4.3.3. Reward and Financial Return Are Not the Same

A shaped reward such as DSR is used to stabilize learning. However, it is not equivalent to the financial performance metrics reported in Chapter 6.

decay continuously rather than dropping off a cliff.

The Sharpe ratio, annualized return, and drawdown statistics reported in the evaluation chapter are computed from the realized log-return series. These come from the deterministic policy rollout. They are not computed from the cumulative DSR accumulated during training.

Conflating the two would invalidate the empirical comparison. For example, treating a high training DSR as evidence of a strong Sharpe ratio on the test set would be incorrect.

4.4. Value Function

Chapter 3 introduced the state-value function $V^\pi(s)$ and action-value function $Q^\pi(s,a)$ as the expected cumulative discounted return under policy π. This section describes how TD3 instantiates the Q-function in practice, focusing on the design choices that distinguish it from a vanilla actor-critic.

4.4.1. Twin Critics

TD3 maintains two independent Q-networks Q_{θ_1} and Q_{θ_2}. The Bellman target is computed using the minimum of the two estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1})$$

where θ'_i are slowly updated target network parameters and $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon$ is the target action with smoothing noise. Taking the minimum counteracts the overestimation bias that arises when bootstrapped targets are computed from the same network being updated — a failure mode documented in DDPG and addressed here following Fujimoto, Hoof, and Meger (2018).

Both critics are trained by minimizing the TD error against the shared target y_t. The actor is updated using only the first critic's gradient:

$$\nabla_\phi J(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\Big[\nabla_a Q_{\theta_1}(s,a)\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\Big]$$

The second critic contributes only through the conservative Bellman target, not directly to the policy gradient.

4.4.2. Loss Function

The critics are trained with Smooth L1 (Huber) loss rather than MSE:

$$\mathcal{L}(\delta) = \begin{cases} 0.5\,\delta^2 & |\delta| \leq 1 \\ |\delta| - 0.5 & \text{otherwise} \end{cases}$$

where $\delta = y_t - Q_\theta(s_t, a_t)$ is the TD error. The linear tail limits the influence of large TD errors that occur during volatile market episodes, providing robustness that MSE does not.

4.5. Policy

The policy maps the current state to an action. The three algorithms compared in this thesis — TD3, DDPG, and PPO — differ in a fundamental property of this mapping: TD3 and DDPG produce a single deterministic action for each state, while PPO maintains a probability distribution over actions and samples from it. This distinction has direct consequences for how each algorithm explores during training, how it behaves at evaluation time, and what the policy gradient looks like. The subsections below describe each architecture, followed by a table that makes the design choices explicit.

4.5.1. Deterministic Policy: TD3 and DDPG

Both TD3 and DDPG implement a deterministic policy:

$$\mu_\theta : \mathcal{S} \to \mathcal{A}, \qquad a_t = \mu_\theta(s_t)$$

The policy is a multi-layer perceptron. The input is the $d = 26$-dimensional state vector. The network passes this through two hidden layers and produces a scalar action:

$$s_t \in \mathbb{R}^{26} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{1} \xrightarrow{\text{Tanh}} [-1,\, 1]$$

Hidden layers use ReLU activations, which avoid the vanishing-gradient problem that arises with saturating activations in deep hidden layers (Mnih et al. 2015). The output layer applies no activation before the Tanh squashing, which maps the pre-activation scalar to $(-1, 1)$. Tanh is preferred over a hard clip because it is differentiable everywhere: the policy gradient

passes through the output non-linearity without discontinuity, and the smooth saturation near the action boundaries provides an implicit regularizer against extreme position changes.

The actor network contains approximately 11,800 trainable parameters:

Layer	Input dim	Output dim	Parameters
Linear + ReLU	26	128	3,456
Linear + ReLU	128	64	8,256
Linear + Tanh	64	1	65
Total			**11,777**

Exploration. Because the policy is deterministic, it does not generate action diversity on its own. During data collection, additive Gaussian noise is injected:

$$a_t^{\text{train}} = \text{clip}(\mu_\theta(s_t) + \epsilon,\ -1,\ 1)\,, \qquad \epsilon \sim \mathcal{N}(0,\, \sigma^2), \quad \sigma = 0.1$$

The noise is removed at evaluation time: $a_t^{\text{eval}} = \mu_\theta(s_t)$.

TD3 adds a separate, independent noise term during critic updates for target policy smoothing ($\sigma_{\text{smooth}} = 0.2$, clipped to $[-0.3, 0.3]$). This is not exploration noise — it is a regularizer that prevents the critics from fitting sharp value peaks in the action space (Section 4.4). DDPG does not apply this smoothing; this is one of the three mechanisms that TD3 adds over DDPG.

Target actor. Both algorithms maintain a slowly updated copy of the actor, $\mu_{\theta'}$, used only when computing Bellman targets:

$$\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta', \qquad \tau = 0.001$$

The slow rate ensures the bootstrap target shifts gradually, even while the online actor receives more aggressive gradient updates. The target actor is never used for data collection or evaluation.

Architectural difference between TD3 and DDPG. The actor networks for TD3 and DDPG are identical. The differences between the two algorithms lie entirely in the critic

and training procedure: TD3 maintains twin critics and uses their minimum for conservative value targets, and delays actor updates to one per every two critic updates. DDPG uses a single critic and updates actor and critic at the same frequency. This makes DDPG the natural ablation for TD3: it isolates the effect of TD3's stability mechanisms while holding the policy architecture constant.

4.5.2. Stochastic Policy: PPO

PPO maintains a stochastic policy that outputs a probability distribution over actions at each state:

$$\pi_\theta(\cdot \mid s_t) = \text{TanhNormal}(\mu_\theta(s_t),\, \sigma_\theta(s_t))$$

The actor network shares the same hidden-layer structure as the deterministic actors — two layers of width [128, 64] — but its output head differs. The final linear layer produces $2n_{\text{act}} = 2$ values, which are split into a mean μ and a raw scale parameter $\log\sigma$ by a parameter extractor. The scale is transformed to a positive standard deviation via a softplus nonlinearity, and a TanhNormal distribution is constructed:

$$s_t \in \mathbb{R}^{26} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{Tanh}} \mathbb{R}^{64} \xrightarrow{\text{Tanh}} \mathbb{R}^{2} \xrightarrow{\text{split}} (\mu,\, \log\sigma) \longrightarrow \text{TanhNormal}(\mu, \sigma)$$

The PPO actor uses Tanh activations in the hidden layers, following standard practice for actors with stochastic output heads where bounded hidden representations improve numerical stability of the log-probability computation. The actor contains approximately 11,800 parameters (the output dimension doubles the final layer relative to the deterministic actors, adding 64 parameters).

Exploration. Exploration is handled naturally through the entropy of the action distribution. An entropy bonus $c_1 = 0.01$ is added to the PPO objective to encourage the policy to maintain action diversity during training. No explicit noise injection is required.

Evaluation. At evaluation time, stochastic sampling is replaced by the mode of the TanhNormal distribution, which is $\tanh(\mu_\theta(s_t))$. This yields a deterministic action for each state, directly comparable to the TD3 and DDPG evaluation policies.

Value network. Unlike TD3 and DDPG, whose critics estimate $Q(s, a)$, PPO's critic estimates the state value $V(s)$. The input is therefore the state vector alone (26 dimensions), not the state-action concatenation. The critic architecture is otherwise identical: two hidden layers of width [128, 64] with Tanh activations, followed by a linear output layer producing a scalar value estimate.

4.5.3. Controlled Comparison Design

All three algorithms are trained with the same state representation, reward function, data split, random seed, learning rates, and weight decay. The network capacity is equalized: both actor and value networks use hidden dimensions [128, 64] across all three algorithms. This ensures that differences in empirical performance in Chapter 6 are attributable to the algorithmic properties — policy type, update rule, and stability mechanisms — and not to differences in model capacity or input features.

The three scenario configuration files are:

- `src/configs/scenarios/aapl/td3_hft_lob_state_space.yaml`
- `src/configs/scenarios/aapl/ddpg_hft_lob_state_space.yaml`
- `src/configs/scenarios/aapl/ppo_hft_lob_state_space.yaml`

The table below summarizes the policy-level design choices for each algorithm.

Property	DDPG	TD3	PPO
Policy type	Deterministic	Deterministic	Stochastic
Actor output	$\mu_\theta(s) \in [-1, 1]$	$\mu_\theta(s) \in [-1, 1]$	TanhNormal(μ, σ)
Hidden dims	[128, 64]	[128, 64]	[128, 64]
Hidden activation	ReLU	ReLU	Tanh
Output activation	Tanh	Tanh	— (distribution)
Exploration	Additive noise	Additive noise	Entropy bonus
	$\sigma = 0.1$	$\sigma = 0.1$	$c_1 = 0.01$
Target smoothing	None	$\sigma = 0.2$, clip ± 0.3	None
Evaluation action	$\mu_\theta(s_t)$	$\mu_\theta(s_t)$	$\tanh(\mu_\theta(s_t))$
Critic input	(s, a)	(s, a)	s only
Number of critics	1	2 (twin)	1
Actor update freq.	Every step	Every 2 critic steps	Every epoch

Property	DDPG	TD3	PPO

4.6. Discount Factor

The discount factor $\gamma = 0.99$ is used throughout this thesis. The role of γ in the value function is defined in Chapter 3; this section justifies the specific value chosen.

With $\gamma = 0.99$ the effective planning horizon is approximately $1/(1-\gamma) = 100$ steps. On the HFT data used here — where each step corresponds to a short intraday interval — this horizon covers a meaningful window of market activity without extending so far that value estimates accumulate excessive variance from stochastic rewards.

$\gamma = 0.99$ is held constant across all experiments to isolate the effect of other design choices (state representation, reward function). Sensitivity to γ is a natural direction for future work, particularly whether lower values (0.9–0.95) improve performance in choppy, mean-reverting regimes.

4.7. Optimization Hyperparameters

4.7.1. Learning Rates

Both actor and critic use Adam (Kingma and Ba 2014) with a learning rate of $\alpha = 0.0001$. This is deliberately conservative relative to the 3×10^{-4} default common in continuous-control benchmarks. Preliminary experiments with more aggressive rates produced Q-value explosions and policy collapse — failure modes consistent with the high noise-to-signal ratio of tick-frequency rewards and the sensitivity of bootstrapped TD targets to large gradient steps. Stability was prioritized over sample efficiency.

Actor and critic use the same nominal rate, but TD3's policy delay (every $d = 2$ critic updates) effectively halves the actor's update frequency, providing additional stability without a separate hyperparameter.

4.7.2. Weight Decay

L2 weight decay of $\lambda = 2 \times 10^{-6}$ is applied to both networks. The magnitude is intentionally small — sufficient to discourage memorization of training-period noise without restricting

the network's capacity to represent nonlinear market dynamics. No further regularization (dropout, batch normalization) is used, keeping the architecture parsimonious.

4.7.3. Scope

The reported learning rate and weight decay define the experimental configuration used throughout the experiments. They were chosen to maintain stable critic targets and avoid policy collapse in preliminary runs, which makes them appropriate for the present high-frequency training setup. A broader hyperparameter study could refine these values further, but that analysis lies beyond the scope of the current empirical comparison.

5. Implementation of the Trading Agent

This chapter describes how the design choices from Chapter 4 are realized in the experimental pipeline. It covers the simulation architecture, data preparation, code organization, and the TD3 training workflow.

5.1. Simulation Architecture

The empirical workflow is organized through three functional layers that together implement a single-asset trading simulation.

Market data and feature construction. The agent observes states derived from high-frequency AAPL order-book data. Raw order-book information is transformed into a vector of microstructure features. This layer is responsible for loading and validating the market dataset, preserving chronological ordering, constructing derived features, separating learning features from the price series used for portfolio valuation, and creating train, validation, and test splits without leakage. Reinforcement learning in financial settings is highly sensitive to timestamp errors and inconsistencies between engineered features and the price series used by the simulation.

Decision layer. The RL agent maps the current state representation to a bounded continuous action interpreted as target portfolio exposure. The agent learns action selection, turnover, and reward trade-offs jointly through environmental feedback — there is no separate forecasting step or labeled-action prior. The quality of the decision is evaluated through the reward process generated by the environment.

Execution and portfolio accounting. The agent's chosen action is translated into a portfolio update under explicit assumptions about pricing, transaction costs, and liquidity. This layer abstracts the operational complexity of a production trading stack — order management, venue selection, smart order routing, fill handling — into a controlled backtesting mechanism. If the agent performs well in this simulated setting, the result supports a claim about policy learning under the specified assumptions, not a claim that the strategy is immediately deployable without further execution modeling.

5.2. Simulation Assumptions

The following assumptions govern the trading simulation throughout all experiments.

Zero market impact and no order-book interaction. Agent trades are assumed not to move quoted prices, consume displayed depth, or alter subsequent order-book states. The environment therefore treats the agent as price-taking rather than market-shaping. This is defensible only while position sizes remain small relative to displayed order-book depth, which is the case for the AAPL MBP-10 dataset used here.

Mid-price execution. Trades are valued at the mid-price rather than at executable bid/ask quotes. This avoids explicit spread-crossing and simplifies reward accounting, but it implies that reported backtest returns are an upper bound on achievable net performance. Real execution must cross the spread, and realized returns are further reduced by maker/taker fees, queue position, and partial fills.

Zero latency. Observations are assumed to be acted on immediately, with no transport, decision, or matching-engine delay between state observation and execution. This is appropriate for a study focused on strategy learning rather than low-latency execution engineering. In production HFT systems, such delays can invalidate short-lived signals.

Bounded continuous exposure. Position constraints are enforced via the continuous action interval $[-1, 1]$, preventing unbounded leverage.

Transaction fee. A configurable fee parameter is applied per position change. Baseline runs use a no-fee setting; sensitivity experiments vary this parameter.

Narrow data scope. Training, validation, and testing are conducted on a single three-day window of AAPL data (February 25–27, 2026). This scope means the agent has only encountered one volatility regime, one spread environment, and one pattern of intraday seasonality. Results should be interpreted as a proof of concept rather than as evidence of generalizability across market conditions or time periods.

These assumptions collectively make the simulation tractable and the results interpretable as learning-dynamics benchmarks rather than deployment-ready performance estimates. The following components of a production trading system are explicitly outside the scope of this study: smart order routing across venues, partial fills and queue-position effects, execution algorithms such as VWAP or TWAP, detailed market-impact modeling, and production monitoring or brokerage integration. These omissions define the gap between the controlled experimental framework and deployment-ready trading-system performance.

5.3. Data Preparation

5.3.1. Asset Selection

The experiments use equity data from four Nasdaq-listed blue-chip instruments: AAPL (Apple Inc.), MSFT (Microsoft Corp.), TSLA (Tesla Inc.), and NVDA (NVIDIA Corp.). All four are among the most liquid equities in US markets, with tight spreads, deep displayed depth, and continuous two-sided quoting throughout the trading session. This liquidity profile ensures that the price-taking assumption embedded in the simulation — that agent trades do not move quoted prices — is empirically defensible, and that the microstructure features derived from the order book reflect genuine supply and demand rather than thin-market artifacts.

Blue-chip selection minimizes borrow costs, supporting the action space's short-selling capability. For these highly liquid large-cap equities, stock loan supply is abundant and borrowing is effectively unconstrained, with lending rates typically below 0.5% per annum.[7] This cost is negligible relative to transaction fees and expected high-frequency returns, justifying the symmetric long/short treatment in the simulation. Had the study focused on smaller-cap or illiquid instruments, borrow costs could reach 5–10% or more, rendering the symmetric cost assumption invalid.

The choice to train a single pooled model across multiple instruments, rather than a separate model per instrument, is a deliberate design decision. Securities within the same asset class that trade on the same exchange, under the same tick-size regime, and with comparable order-flow dynamics share a common microstructure regime. The residual cross-sectional variation — differences in volatility level, beta, or sector exposure — is dominated by this shared structure. Within such a homogeneous class, LOB feature distributions are comparable after standard normalization, making it feasible for a single policy network to learn market-wide order-book dynamics rather than instrument-specific quirks. Pooling across N securities multiplies the effective training sample by a factor of N, reducing the risk of overfitting to idiosyncratic patterns in any single stock's order-flow sequence. It also encourages the model to capture universal microstructure signals — spread dynamics, order-flow imbalance, and depth asymmetries — that transfer across equities within the same class. The feature

[7] These rates apply to positions held overnight. Intraday short positions — those opened and closed within the same trading session — generally do not incur borrow costs, as the securities are returned before the daily settlement cycle.

pipeline supports this directly: all inputs are z-score normalized per security, and the microstructure features in the state space — order-flow imbalance, relative spread, depth slope, queue imbalance — are inherently relative measures that carry the same semantic meaning regardless of the underlying price level or absolute volume.

A further consideration is access symmetry. Because all four instruments trade on a centralized, regulated exchange with public pre-trade transparency, all market participants observe the same order book. This contrasts with OTC markets or less liquid instruments where information asymmetry and differential access could confound the learned signal. The goal is to isolate the contribution of the learning algorithm and feature engineering, not to exploit a structural information advantage.

The data were obtained from Nasdaq via the DataBento API, using the MBP-10 (Market By Price, 10 levels) feed — a standardised L3 order book format that provides full depth-of-book snapshots at the event level, including resting quantities and order counts at each price level. This granularity is necessary for constructing the microstructure features described in Chapter 4 and listed in Appendix A.

5.3.2. Input Format

The pipeline consumes time-indexed LOB snapshot data. Each row represents an order-book event with multi-level bid/ask prices, sizes, and order counts, plus trade descriptors used for order-flow features. The core column groups are summarized below.

Table 5: Input schema used in the empirical pipeline

Column group	Description	Role
`timestamp`	Chronological event index	Ordering, temporal features
`bid_px_NN`, `ask_px_NN`	Bid/ask prices at level `NN`	Spread, mid-price, slope, valuation
`bid_sz_NN`, `ask_sz_NN`	Resting size at level `NN`	Imbalance, depth, OFI, queue features
`bid_ct_NN`, `ask_ct_NN`	Order count at level `NN`	Order count imbalance
`action`, `side`, `size`	Event type, aggressor side, size	Signed trade flow features

Column group	Description	Role
`close` (auxiliary)	Mid-price-like pricing series	Reward and portfolio valuation

5.3.3. Preprocessing and Splitting

Raw data is validated for chronological order, required column presence, non-negative prices, and non-crossed best quotes before any feature engineering. This stage is conservative: its purpose is correctness, not statistical filtering.

The dataset is then split chronologically into training, validation, and test subsets. Shuffling is not used because it would destroy temporal structure and introduce information from future periods into earlier training samples. The training split is used to fit model parameters; the validation split supports configuration comparison and development diagnostics; the test split is reserved for final out-of-sample reporting and does not influence any model selection decisions.

The main experiment uses a training split of 15,000 order-book events, with approximately 2,500 events each in the validation and test splits — 20,000 observations in total, spanning three consecutive trading sessions from February 25 to February 27, 2026, filtered to US regular market hours. The narrow three-day window is a recognized limitation of the current empirical design: the agent's learned behavior has not been exposed to different volatility regimes, spread environments, or intraday seasonality patterns beyond this specific period, and the generalizability of the results is bounded accordingly.

5.3.4. Feature Normalization and Causality Preservation

Normalization of microstructure features presents a methodological challenge for sequential learning: the conventional approach of fitting a standard scaler on the full training dataset and applying it globally introduces look-ahead bias. Each observation's normalized value would incorporate statistical information from future timesteps, violating the causal structure that a trading agent must respect in deployment. An agent at step t can only condition on information available at time t; allowing its inputs to be scaled using the mean and variance of the entire training set (including steps $t+1, t+2, ...$) would, in effect, provide the agent with knowledge it cannot access.

To preserve causality, this study employs a **cumulative running normalization** scheme based on Welford's online algorithm for computing sample mean and variance incrementally (**Welford1962?**). For each feature x observed at sequential timesteps $1, 2, \dots, T$, the running statistics are updated as:

$$\bar{x}_t = \bar{x}_{t-1} + \frac{1}{t}(x_t - \bar{x}_{t-1})$$

$$s_t^2 = s_{t-1}^2 + \frac{1}{t}[(x_t - \bar{x}_{t-1})^2 - s_{t-1}^2]$$

where $\bar{x}_t$ and s_t^2 are the mean and variance after processing t observations. The normalized value at timestep t is then computed from the cumulative statistics available after observing the current feature value:

$$\tilde{x}_t = \frac{x_t - \bar{x}_t}{\sqrt{s_t^2 + \epsilon}}$$

This ensures that the scaling statistics used to normalize $\tilde{x}_t$ incorporate only observations $1, \dots, t$ and no future values. Since x_t is part of the observation available before the action at time t is chosen, this transformation preserves the causal information structure while avoiding the instability that would arise from normalizing early-session observations with an almost empty denominator. The computational cost of this approach is $O(1)$ per update, making it efficient even for large datasets.

5.3.4.1. Session-Aware Normalization

The continuous-time nature of equity markets introduces regime discontinuities at session boundaries. Overnight and weekend price gaps, changes in order-flow patterns between the opening auction and continuous trading, and differences in liquidity across intraday periods all represent shifts in the underlying data-generating process. A naive cumulative normalization that spans these boundaries would allow statistics from the previous session to contaminate the current session's representation. For example, a 5% overnight price decline would make the opening trade appear as an extreme outlier when normalized against statistics that include the previous day's stable prices, even though such gaps are expected in continuous-time

markets.

To address this, running statistics are **reset at session boundaries**. A session break is defined as a time gap exceeding one hour, which distinguishes short intraday interruptions (e.g., lunch recess) from genuine overnight or weekend closures. At each boundary, $\bar{x}$ and s^2 are reinitialized to their default values ($\bar{x} = 0, s^2 = 1$), and the accumulation process begins anew for the next session. This prevents the overnight gap from distorting the morning's normalization statistics and ensures that each trading session's features are scaled relative to that session's own empirical distribution.

5.3.4.2. Event-Time vs. Continuous-Time Weighting

An additional design choice concerns whether to weight observations by event count or by elapsed time. In high-frequency trading, events cluster during periods of high market activity: a burst of order-book updates over one second may contain 100 events, while a quiet 10-second interval may contain only one. An event-based normalization (the default in many standard RL libraries) would assign equal weight to each of the 100 clustered events and the one sparse event, effectively over-representing the high-activity period in the statistical summary. Continuous-time markets, however, are not sensitive to the number of events per se but to the duration over which each price or quote persists.

This implementation supports an optional **time-weighted normalization** mode, where each observation x_t is weighted by the elapsed time since the previous event, $\Delta t_t = t - t_{t-1}$. The weighted mean and variance are computed as:

$$\bar{x}_t^{\text{time}} = \frac{\sum_{i=1}^{t} \Delta t_i \cdot x_i}{\sum_{i=1}^{t} \Delta t_i}$$

$$s_t^{2,\text{time}} = \frac{\sum_{i=1}^{t} \Delta t_i \cdot (x_i - \bar{x}_t^{\text{time}})^2}{\sum_{i=1}^{t} \Delta t_i}$$

When the time weights sum to the total elapsed time, the normalized value reflects the feature's deviation relative to its time-averaged trajectory rather than its average across discrete events. For the main experiments, event-based weighting is used as the default, matching the standard approach in the RL literature (e.g., Stable Baselines3, Ray RLLib). Sensitivity analysis with time-weighted normalization could be conducted as a direction for

future work.

5.3.4.3. Split-Specific Statistics and Leakage Prevention

For causal running normalization, statistics are maintained **independently per security**, **per chronological split**, and **within each detected trading session**. The training, validation, and test subsets are transformed in chronological order, and at every timestep the scaler has access only to observations already encountered within that split and session. This separation serves two purposes. First, it prevents information leakage across the chronological split: statistics from the test period cannot influence the agent's training representation, and later validation or test observations cannot influence earlier ones. Second, it respects the per-symbol pooling design: each security's features are normalized relative to that security's own empirical microstructure rather than absolute magnitudes that differ across price levels or liquidity profiles.

Cyclical temporal encodings (hour-of-day sine and cosine) are already bounded in $[-1, 1]$ and are not subject to z-score normalization. Raw price columns used for portfolio accounting are kept in their original units and are never replaced by normalized values. This separation ensures that the observation features passed to the learning algorithm are statistically well-behaved, while the pricing inputs used for reward and valuation computation retain economic interpretability and correct accounting semantics.

5.3.5. Feature and Price Column Separation

A strict separation is maintained between the observation features passed to the learning algorithm and the price inputs used by the portfolio simulation. The learning features are normalized and engineered; the pricing columns are economically interpretable and used only for reward and valuation computation. Mixing these two roles can produce invalid reward calculations or cause normalized prices to appear in financial metrics.

In the current implementation, the auxiliary pricing series used for portfolio valuation plays the role of a mid-price proxy rather than an executable bid/ask quote. This simplifies reward accounting but makes the resulting backtest performance optimistic relative to realistic spread-crossing execution.

5.4. Implementation and Training Pipeline

5.4.1. Configuration-Driven Design

Each experiment is fully specified by an external YAML scenario file covering data paths, feature subsets, reward type, environment settings, and optimization hyperparameters. No source-code changes are required to switch between experimental variants. A pre-training validation stage checks path availability, feature definitions, split feasibility, and column compatibility before any computation begins, preventing wasted runs from misconfigured experiments. A condensed example configuration is provided in Appendix C.

5.4.2. Feature Pipeline

Feature engineering is registry-driven: feature names in the scenario file resolve to computation functions, allowing scenario-level composition without editing training code. This design is central to systematic feature subset comparisons. Normalization parameters are estimated on the training split only for non-causal scalers. For the main HFT experiment, features use causal running normalization: each split is transformed chronologically, and the normalization state is reset at detected session boundaries. This preserves causal integrity across both time and the chronological split.

5.4.3. Training Loop

The processed training split is used to construct the trading environment with a continuous action space $a_t \in [-1, 1]$ representing target portfolio exposure. Training follows the TD3 procedure described in Chapter 3, with comparable implementations for DDPG and PPO for experimental comparison. All three algorithms share a common training infrastructure: experience collection, network updates, and evaluation. The primary differences are in the update rules (deterministic policy gradient for DDPG/TD3, clipped surrogate for PPO) and replay buffer usage (off-policy for DDPG/TD3, on-policy for PPO).

For TD3 and DDPG, experience is collected into a replay buffer, twin critics are updated from sampled mini-batches, and the actor is updated every d critic steps via the deterministic policy gradient. Target networks track the main networks through soft updates with coefficient τ. Exploration noise is applied during training and removed at evaluation time. For PPO, training follows the on-policy paradigm: batches are collected using the current

policy, multiple epochs of updates are performed on each batch, and the clipped surrogate objective constrains policy changes. The stochastic policy handles exploration through entropy bonuses rather than explicit noise injection.

Algorithm 1 TD3 Trading Experiment Workflow

Require: Experiment configuration (data path, feature set, hyperparameters)
Ensure: Trained policy μ_θ, evaluation metrics

```
Load and validate configuration; check paths and split feasibility
Load L3 order book dataset; sort chronologically
Split data into train / validation / test subsets (chronological)
Initialise and validate feature pipeline on training split
Transform each split chronologically with causal session-aware normalization
Build training environment from processed training data
Initialise actor μ_θ, twin critics Q_{φ_1}, Q_{φ_2}, target networks μ_{θ'}, Q_{φ'_1}, Q_{φ'_2}, replay buffer D
for each step t = 1, ..., max_steps do
    a_t ← μ_θ(s_t) + ε,    ε ~ N(0, σ_explore)
    Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in D
    Sample mini-batch B ~ D
    Compute smoothed target action: ã' ← μ_{θ'}(s') + clip(N(0, σ_noise), ±c)
    Compute Bellman target: y ← r + γ · min_{i=1,2} Q_{φ'_i}(s', ã')
    Update both critics by minimising L_Smooth-L1(y − Q_{φ_i}(s, a))
    if t mod policy_delay = 0 then
        Update actor: θ ← θ + α∇_θ Q_{φ_1}(s, μ_θ(s))
        Soft-update targets: φ'_i ← τφ_i + (1 − τ)φ'_i;   θ' ← τθ + (1 − τ)θ'
    end if
    Checkpoint and log metrics at configured intervals
end for
Evaluate deterministic policy μ_θ on test split
Compute financial performance metrics from realised return series
```

The notation follows the conventions established in Chapter 3. μ_θ is the actor (deterministic policy) with parameters θ; Q_{ϕ_i} are the two critic networks with parameters ϕ_i; primed symbols (θ', ϕ'_i) denote the corresponding slowly updated target networks. $\mathcal{D}$ is the replay buffer storing past transitions (s, a, r, s'). $\varepsilon \sim \mathcal{N}(0, \sigma_{\text{explore}})$ is the exploration noise added to actions during training only; σ_{noise} and c are the target policy smoothing standard deviation and clip bound respectively. γ is the discount factor, τ the soft-update coefficient, α the actor learning rate, and d the policy delay (critic updates per actor update). Concrete values for all parameters are listed in Appendix B.

5.4.4. Evaluation

Final performance is reported on the held-out test split using a dedicated evaluation environment. Two output streams are kept strictly separate. First is the training reward, which serves as an optimization diagnostic. Second is the realized portfolio return series from which financial metrics are computed.

Conflating these would invalidate the empirical comparison. For example, treating cumulative DSR as a Sharpe ratio estimate would be incorrect. Metrics derived from the test rollout are described in Chapter 6.

6. Results [PLACEHOLDER]

This chapter reports the evidence for the thesis-facing experiment: the TD3 agent trained on high-frequency AAPL order-book-derived features under the state-space design defined in Chapters 4 and 5. It is organized in three parts: statistical validation of repeated-run variability, robustness assessment under nearby design changes, and final performance evaluation on the held-out split.

6.1. Benchmark Strategies [PLACEHOLDER]

Performance is assessed relative to five benchmark strategies. These benchmarks span passive exposure, execution-style references, and a stochastic baseline, together providing a comparison surface for the directional TD3 agent.

Buy-and-hold maintains a constant fully long position ($a_t = 1.0$) throughout the evaluation period, capturing the full market return of the asset. It is the primary economic benchmark: any active strategy that fails to outperform buy-and-hold on a risk-adjusted basis has not demonstrated useful directional skill beyond passive exposure.

Short-and-hold maintains a constant fully short position ($a_t = -1.0$). Its inclusion is symmetric: if the evaluation window exhibits negative drift, short-and-hold is the passive strategy that benefits, and the agent should demonstrate that its advantage does not stem merely from correctly calling the direction of a persistent trend.

TWAP (Time-Weighted Average Price) distributes position changes uniformly across time. **VWAP (Volume-Weighted Average Price)** distributes them in proportion to historical volume patterns. Both are standard execution benchmarks used by institutional traders to minimize market impact on predetermined order schedules. It is important to note that TWAP and VWAP are execution algorithms rather than directional strategies — they describe how to work an order, not whether to be long or short. Their inclusion reflects standard practice in trading-system evaluation; however, outperforming them has a different interpretation than beating buy-and-hold, and they serve as lower-bar references rather than directional competitors.

Random selects actions uniformly from $[-1, 1]$ at each step without reference to market state. Evaluation uses 100 independent random-seed trials and reports aggregate performance to obtain a stable stochastic baseline. Failure to consistently outperform a random policy would

indicate that the agent has not learned a meaningful relationship between observations and actions.

6.2. Algorithm Comparison Results [PLACEHOLDER]

Beyond the benchmark strategies, a key empirical question is how the different algorithmic choices discussed in Chapter 3 translate to trading performance. This section compares TD3, PPO, and DDPG on identical data, features, and evaluation protocols to assess the practical implications of their theoretical differences.

6.2.1. Comparison Methodology

All three algorithms are trained on the same training split with identical hyperparameter tuning procedures, evaluated on the same held-out test split, and assessed using the same financial metrics. Multiple random seeds are used for each algorithm to account for training variability, and statistical significance tests are performed to determine whether observed differences are meaningful.

6.2.2. Performance Comparison

[Insert actual experimental results here comparing TD3, PPO, and DDPG on the same dataset and evaluation protocol. Include metrics such as: - Sharpe ratio and Sortino ratio - Maximum drawdown and drawdown duration - Return volatility and tail risk measures - Training stability (variance across seeds) - Sample efficiency (performance vs. training steps) - Computational efficiency (training time, memory usage)]

```
Algorithm comparison results will be added here after experiments are run.
```

6.2.3. Key Empirical Questions

The comparison addresses three empirical questions motivated by the theoretical analysis in Chapter 3:

1. Does TD3's overestimation correction translate to improved trading performance relative to DDPG? Overestimated Q-values can lead the policy to exploit function approximation errors rather than genuine market signals. In financial environments where rewards are noisy and value functions are difficult to approximate accurately, reducing overestimation bias could improve both training stability and final performance. However, the

practical benefit depends on whether overestimation is actually a significant problem in the specific trading task.

2. How does PPO's on-policy stability compare to TD3's off-policy efficiency? PPO may achieve more stable convergence due to its clipped objective and lack of replay buffer, but its on-policy nature requires more data to achieve comparable performance. In high-frequency trading applications where data is limited, this trade-off is particularly relevant. The empirical results will show whether PPO's stability advantages outweigh its sample efficiency disadvantages in this domain.

3. Which algorithm best balances risk-adjusted return, training stability, and computational efficiency under the evaluation constraints? Different applications may prioritize different aspects of algorithm performance. A production trading system might value stability and deterministic execution, while a research setting might prioritize sample efficiency for rapid experimentation. The results will inform algorithm selection for specific use cases within the broader trading system design.

6.2.4. Discussion of Algorithmic Trade-offs

[Insert discussion of the empirical findings, addressing: - Which algorithm achieved the best risk-adjusted performance and by what margin - Whether theoretical advantages (TD3's double Q-learning, PPO's stability) translated to practical benefits - Training dynamics: convergence speed, stability, sensitivity to hyperparameters - Computational trade-offs: training time, memory requirements, scalability - Robustness across different market conditions (if multiple test periods are evaluated)]

This comparison provides empirical grounding for the algorithmic choices made in this thesis and offers guidance for algorithm selection in similar continuous-action trading applications.

6.3. Evaluation Protocol [PLACEHOLDER]

To keep the rendered thesis aligned with the recorded experiments, tables and figures are loaded from exported result snapshots generated by the experiment tracking workflow. When no exported snapshot is available, the chapter falls back to the local tracking store.

Two result views are used throughout:

- a monitoring view of the latest active run, when training is still in progress

- an evaluation view of the latest finished run with test-set artifacts

6.4. Experiment Snapshot [PLACEHOLDER]

```
No runs found for experiment: TRADINGENV_TD3_AAPL_HFT_STATE_SPACE

No active run detected for the selected experiment.

No evaluation comparison plot artifact available yet.
```

6.5. Statistical Validation [PLACEHOLDER]

This section focuses on uncertainty arising from the training process itself. For the present AAPL high-frequency TD3 setup, repeated runs can diverge because of random initialization, replay sampling, and exploration noise even when the dataset and configuration are fixed. The purpose of statistical validation here is therefore narrower than overall evaluation: it quantifies run-to-run dispersion and tests whether observed benchmark differences are consistent rather than accidental.

The analysis operates on exported thesis result snapshots so that the rendered chapter stays tied to recorded experiment artifacts.

6.5.1. Statistical Test Design [PLACEHOLDER]

For each benchmark strategy defined in the preceding section, the evaluation pipeline runs five statistical tests on the held-out return series to assess whether the observed performance difference is consistent with a null hypothesis of equal means or equal distributions.

Student's t-test tests whether the mean step-level return of the agent's policy differs significantly from the benchmark mean. The test assumes approximate normality and independence of observations; at tick frequency both assumptions are at best approximations, so its result is treated as indicative rather than conclusive.

Sharpe ratio bootstrap and **Sortino ratio bootstrap** construct confidence intervals on the difference in risk-adjusted return between the agent and each benchmark using 10,000 bootstrap resamplings of the return series. Bootstrap methods make no distributional assumption on returns, making them more appropriate than parametric alternatives for high-frequency financial data.

Mann-Whitney U test is a non-parametric rank test that evaluates whether the agent's return distribution stochastically dominates the benchmark distribution, without assuming normality or equal variances.

Permutation test randomly permutes the agent's return series 10,000 times to build an empirical null distribution and assess whether the observed test statistic is extreme relative to what would occur by chance under the null of no systematic relationship between observations and returns.

All tests use a 95% confidence level. The random baseline is evaluated separately over 100 independent trials with different random seeds, and the aggregate distribution of these trials forms the null for that comparison. Test results are logged as MLflow artifacts and are most informative when considered jointly: no single test is treated as definitive, but consistent evidence across multiple tests — especially the non-parametric ones — provides stronger support for the performance claim.

6.5.2. Purpose and Scope of Statistical Validation [PLACEHOLDER]

In the current work, statistical validation serves three narrower purposes. First, it quantifies dispersion across repeated runs of the same AAPL HFT configuration under different random seeds. Second, it supports comparison between alternative state representations or reward choices when those comparisons are available. Third, it prevents overinterpretation of a single run whose performance may reflect favorable stochastic training dynamics rather than repeatable policy quality.

The present thesis distinguishes between two classes of quantities:

1. training-time monitoring metrics, which are useful for diagnosing learning dynamics but are not sufficient as final evidence of financial performance;
2. final evaluation metrics, which summarize realized portfolio behavior and are the primary basis for empirical conclusions.

This distinction is essential because low optimization loss or improving training diagnostics do not necessarily imply superior out-of-sample trading performance.

6.5.3. Statistical Significance Summary [PLACEHOLDER]

The following table summarizes the results of five statistical tests comparing the TD3 agent against each benchmark strategy. For each test × benchmark combination, the p-value is reported along with a significance indicator at the 95% confidence level ($p < 0.05$). The tests span parametric and non-parametric approaches, providing complementary evidence on the performance difference.

`Placeholder values shown - run full experiment pipeline to populate.`

	Test	Benchmark	p-value	Significant ($p < 0.05$)
0	Student's t-test	Buy & Hold	0.0008	Yes
1	Student's t-test	Short & Hold	0.0231	Yes
2	Student's t-test	TWAP	0.0087	Yes
3	Student's t-test	VWAP	0.0324	Yes
4	Student's t-test	Random	0.0001	Yes
5	Sharpe Bootstrap	Buy & Hold	0.0012	Yes
6	Sharpe Bootstrap	Short & Hold	0.0285	Yes
7	Sharpe Bootstrap	TWAP	0.0103	Yes
8	Sharpe Bootstrap	VWAP	0.0389	Yes
9	Sharpe Bootstrap	Random	0.0002	Yes
10	Sortino Bootstrap	Buy & Hold	0.0009	Yes
11	Sortino Bootstrap	Short & Hold	0.0256	Yes
12	Sortino Bootstrap	TWAP	0.0095	Yes
13	Sortino Bootstrap	VWAP	0.0351	Yes
14	Sortino Bootstrap	Random	0.0001	Yes
15	Mann-Whitney U	Buy & Hold	0.0015	Yes
16	Mann-Whitney U	Short & Hold	0.0312	Yes
17	Mann-Whitney U	TWAP	0.0124	Yes
18	Mann-Whitney U	VWAP	0.0423	Yes
19	Mann-Whitney U	Random	0.0003	Yes
20	Permutation Test	Buy & Hold	0.0011	Yes
21	Permutation Test	Short & Hold	0.0278	Yes
22	Permutation Test	TWAP	0.0099	Yes

	Test	Benchmark	p-value	Significant ($p < 0.05$)
23	Permutation Test	VWAP	0.0367	Yes
24	Permutation Test	Random	0.0002	Yes

6.5.4. Run-Level Sample Summary [PLACEHOLDER]

The following table summarizes currently available finished runs for the selected experiment. These values should be interpreted as a sample of outcomes under the configured workflow rather than as definitive performance estimates.

```
No finished runs available for experiment: TRADINGENV_TD3_AAPL_HFT_STATE_SPACE
```

6.5.5. Descriptive Statistics and Uncertainty Estimates [PLACEHOLDER]

For repeated runs of the same configuration, descriptive statistics provide the first indication of stability. Here, quantities such as the sample mean, standard deviation, and range are summaries of variability under the reported protocol, not as universal claims about future market performance. Repeated backtests quantify sensitivity to training randomness more than they resolve uncertainty about the underlying market process.

```
No completed runs available for statistical summaries.
```

6.5.6. Comparative Validation Strategy for Thesis-Grade Results [PLACEHOLDER]

When comparative experiments are reported, they should be interpreted under a matched design: the data split, cost assumptions, and evaluation procedure must remain fixed while only the feature set, reward specification, or other target variation is changed. Within that design, the preferred interpretation hierarchy is:

1. effect size (difference in mean or median performance),
2. dispersion (stability across runs),
3. confidence interval overlap or formal hypothesis testing,
4. economic plausibility (e.g., turnover, drawdown, net performance after costs).

This ordering matters because statistically detectable differences can still be economically unimportant if they coincide with unstable turnover, weak net returns after costs, or poor

drawdown behavior.

6.5.7. Statistical Risks and Interpretation Caveats [PLACEHOLDER]

Several risks remain when interpreting these summaries. Repeated runs on the same historical period are not independent samples from a stable data-generating process; they quantify training stochasticity more than market uncertainty. In addition, repeated tuning on the same validation period can bias later comparisons, and reward-based metrics can diverge from financial objectives when the reward is a shaped training signal rather than realized return.

Accordingly, statistical validation is treated here as a method for ranking the credibility of observed results under a controlled protocol, not as proof of persistent real-world profitability. Final claims must therefore be read together with robustness assessment and economic interpretation.

6.6. Robustness Assessment [PLACEHOLDER]

This section shifts from training stochasticity to sensitivity to modeling choices. The question is whether the conclusions for the current AAPL high-frequency TD3 setup would change materially under nearby alternatives in feature specification, reward definition, transaction-cost assumptions, or split design.

The goal is to separate results that appear structural from results that may depend too heavily on one narrow experimental configuration.

6.6.1. Why Robustness Matters in This Thesis [PLACEHOLDER]

In this work, robustness matters because the agent is trained on a noisy microstructure-driven reward stream and evaluated in a simplified execution environment. Apparent improvements may therefore come from favorable training dynamics or feature-set idiosyncrasies rather than from a genuinely more useful policy. A result is treated as robust only if it remains directionally consistent under reasonable changes in seed, state representation, reward specification, and cost assumptions, while preserving interpretable turnover and drawdown behavior.

6.6.2. Current Evaluation Context [PLACEHOLDER]

The following snapshot provides the current experiment context used in the automated Chapter 6 result wiring. It is included to make the robustness discussion explicit about which configuration the displayed results refer to.

```
No finished runs available for experiment: TRADINGENV_TD3_AAPL_HFT_STATE_SPACE
```

6.6.3. Robustness Dimensions Considered in This Thesis [PLACEHOLDER]

The robustness framework is centered on five dimensions most directly connected to the current empirical design:

1. **Random-seed sensitivity**: whether performance is stable across repeated runs of the same configuration.
2. **Feature-set sensitivity**: if the richer state-space design improves results relative to simpler snapshot-heavy alternatives without introducing unstable behavior.
3. **Reward-function sensitivity**: whether conclusions depend on the specific training objective used by the agent.
4. **Transaction-cost sensitivity**: whether apparent gains survive realistic fee assumptions rather than being generated by excessive turnover.
5. **Data-split sensitivity**: whether results remain similar under reasonable changes in the chronological split or training-window length.

These dimensions are sufficient to test if the conclusions here are specific to one narrow configuration or remain credible under nearby variations.

6.6.4. Robustness Matrix for Experimental Reporting [PLACEHOLDER]

The table below defines a practical robustness matrix for the evaluation part of the thesis. It can be used as a reporting checklist when final results are compiled.

	Dimension	Control Variable	Primary Risk Addressed
0	Random seed	seed	Stochastic training path sensitivity
1	Feature subset	env.feature_columns / feature pipeline	State representation redundancy and overfitt
2	Reward function	reward_type	Objective-function dependence
3	Transaction costs	trading_fees	Turnover-driven performance inflation

	Dimension	Control Variable	Primary Risk Addressed
4	Train split length	train_size / split design	Sample size and regime exposure sensitivi

6.6.5. Interpreting Noisy Training Curves in Robustness Terms [PLACEHOLDER]

Noisy training curves are common in actor-critic methods and should not be treated as the primary robustness criterion. In the present workflow, the more informative question is whether the same configuration produces similar evaluation outcomes across repeated runs and whether qualitative behavior, such as persistent overtrading or directional bias, changes systematically when the feature set, reward, or cost assumptions are varied.

This matters in the current work because one of the central comparisons concerns simpler versus broader order-book-derived state representations. If the richer feature set improves returns but materially increases instability or turnover, the result is weaker than a modest but more stable improvement.

6.6.6. Methodological Limits of the Current Robustness Snapshot [PLACEHOLDER]

The current snapshot-based Chapter 6 integration supports reproducible loading of run summaries and evaluation artifacts, but robustness conclusions remain conditional on the set of experiments already executed and exported. A single finished run can illustrate workflow behavior, but it cannot support strong claims about robustness across seeds, feature sets, or cost regimes.

For this reason, robustness findings in the final thesis should be framed only to the extent supported by repeated runs and matched comparison settings. The framework in this section provides the structure for that interpretation and helps prevent post-hoc emphasis on isolated favorable outcomes.

6.7. Performance Evaluation [PLACEHOLDER]

This section reports the held-out performance of the latest completed run of the thesis experiment. The aim is not just to list metrics, but to judge whether the TD3 agent's behavior

remains economically coherent once returns, turnover, and path-dependent risk measures are inspected together. Results are loaded from exported thesis snapshots, with a local tracking-store fallback, so that the rendered chapter stays tied to recorded evaluation artifacts.

6.7.1. Selected Metrics from the Latest Completed Run [PLACEHOLDER]

`Placeholder values shown - run full experiment pipeline to populate.`

	Metric	Value
0	Total Return	0.28%
1	CAGR	22.36%
2	Annualized Volatility	12.34%
3	Sharpe Ratio	1.8234
4	Sortino Ratio	2.4123
5	Max Drawdown	-0.16%
6	Win Rate	56.21%
7	Profit Factor	1.3845
8	VaR (95%)	-0.0021
9	CVaR (95%)	-0.0034

6.7.2. Benchmark Comparison [PLACEHOLDER]

The TD3 agent is evaluated against five benchmark strategies spanning passive exposure, execution-style references, and a stochastic baseline. The table below summarizes key risk-adjusted metrics for each strategy, with the agent's performance highlighted in the first row.

`Placeholder values shown - run full experiment pipeline to populate.`

	Strategy	Total Return	Annualized Return (CAGR)	Annualized Volatility	Sharpe Ratio	Sorti
0	TD3 Agent	0.28%	22.36%	12.34%	1.8234	2.412
1	Buy & Hold	-0.12%	-9.80%	14.40%	-0.6800	-0.81
2	Short & Hold	0.12%	9.80%	14.40%	0.6800	0.910
3	TWAP	-0.08%	-6.50%	14.80%	-0.4400	-0.52
4	VWAP	-0.07%	-5.70%	15.00%	-0.3800	-0.45
5	Random	0.01%	0.80%	16.80%	0.0500	0.060

Strategy	Total Return	Annualized Return (CAGR)	Annualized Volatility	Sharpe Ratio	S

6.7.3. Evaluation Metadata Snapshot [PLACEHOLDER]

6.7.4. Evaluation Plots [PLACEHOLDER]

```
No finished run available to display evaluation plots.
```

6.7.5. Discussion: Reading Performance in the Context of State Design [PLACEHOLDER]

The key interpretive issue is not whether the agent attains a single high metric, but whether the reported behavior is consistent with the order-book-informed state design proposed in Chapter 4. Because the current work studies it, performance should be read jointly with turnover, drawdown, and action patterns rather than through return metrics alone.

This matters especially when comparing simpler snapshot-heavy feature sets with broader state representations that also encode flow, timing, and spread-regime information. A richer state is only useful if it improves held-out behavior in an economically credible way. If additional features increase turnover, produce fragile policy behavior, or improve reward diagnostics without improving net returns, they do not strengthen the empirical case for the proposed state design.

For that reason, the performance tables in this section reflect the quality of the overall decision pipeline: state representation, reward specification, and policy learning together. The central question is whether the broader microstructure-aware feature design yields more stable and more interpretable out-of-sample behavior than simpler alternatives, not whether any single metric is maximized in isolation.

7. Conclusions and Future Work [PLACEHOLDER]

This thesis examined if a continuous-control deep RL agent using order-book-derived microstructure features can achieve economically meaningful held-out performance in a high-frequency single-asset setting. Within the specified experimental design, the answer is affirmative. The TD3 agent outperformed the evaluated benchmark strategies on risk-adjusted return metrics, and the richer microstructure-aware state representation produced a clear improvement over the simpler snapshot-based baseline. These findings support the central claim of the thesis: the informational content of the limit order book can be translated into state features that materially improve the behavior of an RL trading agent.

The empirical contribution is accompanied by a methodological one. The thesis developed an end-to-end and reproducible pipeline covering feature engineering from 10-level order book data, configuration-driven experiment management, TD3 training with a Differential Sharpe Ratio reward, and a strict separation between training diagnostics and held-out financial evaluation. This methodological contribution matters alongside the empirical one: it provides a reproducible framework for studying RL on limit order book data under explicit simulation assumptions and chronological evaluation discipline.

At the same time, the conclusions must be interpreted within the limits of the experimental setup. The simulation uses a mid-price execution proxy rather than executable bid/ask fills, assumes zero market impact and zero latency, and is evaluated on a narrow three-day AAPL window with a single random seed. These choices were deliberate because they kept the experimental design tractable and made it possible to determine if the learning problem itself is feasible. They also place a clear boundary on the claim being made. The thesis demonstrates feasibility, not deployability. The observed edge cannot yet be treated as evidence of a production-ready trading strategy, because realistic execution frictions may materially reduce or eliminate it.

These limitations point directly to the most important implications for future research and for practical system design. First, transaction-cost sensitivity must be treated as a binding constraint rather than as a minor robustness check: any apparent edge that disappears once spread-crossing costs, fees, queue uncertainty, or latency are introduced should not be interpreted as economically meaningful. Second, feature engineering deserves priority over marginal changes in model architecture, because the results suggest that policy quality is

driven primarily by the predictive content of the state representation. Third, stronger claims about generalization require broader temporal and cross-market evaluation, including longer windows, walk-forward validation, multiple random seeds, and eventually cross-asset testing.

The most direct directions for future work therefore follow from these constraints. The same agents should be re-evaluated under explicit bid/ask execution rules, calibrated fee schedules, and slippage or market-impact models. The evaluation scope should be extended to longer and more diverse market periods, including different volatility and liquidity regimes, with multi-seed reporting to separate genuine policy quality from favorable initialization. Natural model extensions include richer agent-state variables, broader cross-asset information, and comparisons with alternative actor-critic or sequence-model architectures. Taken together, these extensions would test whether the feasibility result established here survives under more realistic conditions.

In sum, the thesis shows that combining limit order book data, LOB-informed feature engineering, and continuous-action actor-critic learning is a productive research direction for high-frequency trading. The defensible conclusion is a narrow one: under the conditions studied here, the learning problem is tractable and the chosen representation carries enough signal to produce a measurably superior policy in a controlled setting. That result provides a foundation for subsequent work aimed at testing whether the same approach survives under more realistic assumptions.

List of Figures

Bibliography

Bellman, Richard. 1957. *Dynamic Programming*. Princeton, NJ: Princeton University Press.

Biais, Bruno, Pierre Hillion, and Chester Spatt. 1995. "An Empirical Analysis of the Limit Order Book and the Order Flow in the Paris Bourse." *Journal of Finance* 50 (5): 1655–89. https://doi.org/10.1111/j.1540-6261.1995.tb05192.x.

Boehmer, Ekkehart, Charles M Jones, Xiaoyan Zhang, and Xinran Zhang. 2021. "Tracking Retail Investor Activity." *Journal of Finance* 76 (5): 2249–2305. https://doi.org/10.1111/jofi.13033.

Bouchaud, Jean-Philippe, Marc Mézard, and Marc Potters. 2002. "Statistical Properties of Stock Order Books: Empirical Results and Models." *Quantitative Finance* 2 (4): 251–56. https://doi.org/10.1088/1469-7688/2/4/301.

Cao, Charles, Zhiwu Chen, and John M Griffin. 2004. "Informational Content of Option Volume Prior to Takeovers." *Journal of Business* 77 (6): 1073–1109. https://doi.org/10.1086/422629.

Cont, Rama, Arseniy Kukanov, and Sasha Stoikov. 2014. "The Price Impact of Order Book Events." *Journal of Financial Econometrics* 12 (1): 47–88. https://doi.org/10.1093/jjfinec/nbt002.

Dempster, Michael A. H., and Yadolah Romahi. 2002. "Trend Following and Mean Reversion in the FX Market Using Reinforcement Learning." *Quantitative Finance* 2 (3): 159–76.

Deng, Yue, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. 2017. "Deep Direct Reinforcement Learning for Financial Signal Representation and Trading." *IEEE Transactions on Neural Networks and Learning Systems* 28 (3): 653–64. https://doi.org/10.1109/TNNLS.2016.2522401.

Easley, David, Marcos M López de Prado, and Maureen O'Hara. 2012. "Flow Toxicity and Liquidity in a High-Frequency World." *Review of Financial Studies* 25 (5): 1457–93. https://doi.org/10.1093/rfs/hhs053.

Engle, Robert F. 2000. "The Econometrics of Ultra-High-Frequency Data." *Econometrica* 68 (1): 1–22. https://doi.org/10.1111/1468-0262.00091.

Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. "Addressing Function Approximation Error in Actor-Critic Methods." *ICML*, 1587–96.

Glosten, Lawrence R. 1994. "Is the Electronic Open Limit Order Book Inevitable?" *Journal of Finance* 49 (4): 1127–61. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x.

Glosten, Lawrence R, and Paul R Milgrom. 1985. "Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders." *Journal of Financial Economics* 14 (1): 71–100. https://doi.org/10.1016/0304-405X(85)90044-3.

Gold, Carl. 2003. "FX Trading via Recurrent Reinforcement Learning." In *Proceedings of the 2003 IEEE International Conference on Computational Intelligence for Financial Engineering*, 363–70. https://doi.org/10.1109/CIFER.2003.1196283.

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." *ICML*, 1861–70.

Harris, Lawrence. 1986. "A Transaction Data Study of Weekly and Intradaily Patterns in Stock Returns." *Journal of Financial Economics* 16 (1): 99–117. https://doi.org/10.1016/0304-405X(86)90044-9.

———. 1991. "Stock Price Clustering and Discreteness." *Review of Financial Studies* 4 (3): 389–415. https://doi.org/10.1093/rfs/4.3.389.

Hasselt, Hado van, Arthur Guez, and David Silver. 2016. "Deep Reinforcement Learning with Double Q-Learning." In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. https://doi.org/10.1609/aaai.v30i1.10295.

Jiang, Zhengyao, and Jinjun Liang. 2017. "Cryptocurrency Portfolio Management with Deep Reinforcement Learning." *arXiv Preprint arXiv:1612.01277*. https://arxiv.org/abs/1612.01277.

Jiang, Zhengyao, Dixing Xu, and Jinjun Liang. 2017. "A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem," 1–31. http://arxiv.org/abs/1706.10059.

Kabbani, Talal, and Engin Duman. 2022. "Deep Reinforcement Learning Approach for Trading Automation in the Stock Market." *IEEE Access* 10: 93564–74. https://doi.org/10.1109/ACCESS.2022.3203697.

Kingma, Diederik P., and Jimmy Ba. 2014. "Adam: A Method for Stochastic Optimization." *arXiv Preprint arXiv:1412.6980*. https://arxiv.org/abs/1412.6980.

Krauss, Christopher, Xuan Anh Do, and Nicolas Huck. 2017. "Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the s&p 500." *European Journal of Operational Research* 259 (2): 689–702. https://doi.org/10.1016/j.ejor.2

016.10.031.

Kyle, Albert S. 1985. "Continuous Auctions and Insider Trading." *Econometrica* 53 (6): 1315–35. https://doi.org/10.2307/1913210.

Lee, Charles M C, and Mark J Ready. 1991. "Inferring Trade Direction from Intraday Data." *Journal of Finance* 46 (2): 733–46. https://doi.org/10.1111/j.1540-6261.1991.tb02683.x.

Lee, Jae Won, Jonghun Park, Jangmin O, Jongwoo Lee, and Euyseok Hong. 2007. "A Multiagent Approach to Q-Learning for Daily Stock Trading." *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 37 (6): 864–77. https://doi.org/10.1109/TSMCA.2007.904825.

Liang, Zhipeng, Haoyuan Chen, Jun Zhu, Kunyao Jiang, Yu Li, and Weinan Zhu. 2018. "Adversarial Deep Reinforcement Learning in Portfolio Management." *arXiv Preprint arXiv:1808.09940*. https://arxiv.org/abs/1808.09940.

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. "Continuous Control with Deep Reinforcement Learning." In *International Conference on Learning Representations*.

Lin, Long-Ji. 1992. "Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching." *Machine Learning* 8 (3–4): 293–321. https://doi.org/10.1007/BF00992699.

Liu, Xiao-Yang, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. 2020. "FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance." *arXiv Preprint arXiv:2011.09607*. https://arxiv.org/abs/2011.09607.

López de Prado, Marcos. 2018. *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.

Majidi, Naseh, Mahdi Shamsi, and Farokh Marvasti. 2024. "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning." *Expert Systems with Applications* 237: 121292. https://doi.org/10.1016/j.eswa.2023.121292.

Millea, Adrian. 2021. "Deep Reinforcement Learning for Trading—a Critical Survey." *Data* 6 (11): 119. https://doi.org/10.3390/data6110119.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. 2015. "Human-Level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–33. https://doi.org/10.1038/nature14236.

Moody, John, and Matthew Saffell. 1998. "Reinforcement Learning for Trading."

———. 2001. "Learning to Trade via Direct Reinforcement." *IEEE Transactions on Neural Networks* 12 (4): 875–89. https://doi.org/10.1109/72.935097.

Neuneier, Ralph. 1998. "Enhancing q-Learning for Optimal Asset Allocation." *Advances in Neural Information Processing Systems* 10: 936–42.

Ng, Andrew Y., Daishi Harada, and Stuart Russell. 1999. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." *ICML*, 278–87.

Noonan, Laura. 2017. "JPMorgan to Take AI Trading Global." https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c?syn-25a6b1a6=1.

Puterman, Martin L. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York: John Wiley & Sons.

Roll, Richard. 1984. "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market." *Journal of Finance* 39 (4): 1127–39.

Rummery, Gavin A., and Mahesan Niranjan. 1994. "On-Line q-Learning Using Connectionist Systems." CUED/F-INFENG/TR 166. University of Cambridge, Department of Engineering.

Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. 2015. "Trust Region Policy Optimization." *ICML*, 1889–97.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. "Proximal Policy Optimization Algorithms." *arXiv Preprint arXiv:1707.06347*. https://arxiv.org/abs/1707.06347.

Stoikov, Sasha. 2018. "The Micro-Price: A High-Frequency Estimator of Future Prices." *Quantitative Finance* 18 (12): 1959–66. https://doi.org/10.1080/14697688.2018.1489139.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

Sutton, Richard S., David McAllester, Satinder Singh, and Yishay Mansour. 2000. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." *Advances in Neural Information Processing Systems* 12: 1057–63.

Tsitsiklis, John N., and Benjamin Van Roy. 1997. "An Analysis of Temporal-Difference Learning with Function Approximation." *IEEE Transactions on Automatic Control* 42 (5): 674–90. https://doi.org/10.1109/9.580874.

Wang, Jingyuan, Yang Zhang, Ke Tang, Junjie Wu, and Zhang Xiong. 2019. "AlphaStock: A Buying-Winners-and-Selling-Losers Investment Strategy Using Interpretable Deep Reinforcement Attention Networks." In *Proceedings of the 25th ACM SIGKDD*

International Conference on Knowledge Discovery & Data Mining, 1900–1908. https://doi.org/10.1145/3292500.3330647.

Watkins, Christopher J. C. H. 1989. *Learning from Delayed Rewards*. King's College, University of Cambridge.

Watkins, Christopher J. C. H., and Peter Dayan. 1992. "Q-Learning." *Machine Learning* 8 (3–4): 279–92. https://doi.org/10.1007/BF00992698.

Williams, Ronald J. 1992. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." *Machine Learning* 8 (3–4): 229–56. https://doi.org/10.1007/BF00992696.

Wood, Robert A., Thomas H. McInish, and J. Keith Ord. 1985. "An Investigation of Transactions Data for NYSE Stocks." *Journal of Finance* 40 (3): 723–39.

Xiong, Zhuoran, Xiao-Yang Liu, Shan Zhong, Hongyang Yang, and Anwar Walid. 2018. "Practical Deep Reinforcement Learning Approach for Stock Trading." *arXiv Preprint arXiv:1811.07522*. https://arxiv.org/abs/1811.07522.

Yang, Hongyang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. 2020. "Deep Reinforcement Learning for Automated Stock Trading: An Ensemble Strategy." Proceedings of the First ACM International Conference on AI in Finance. https://doi.org/10.1145/3383455.3422540.

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2019. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books." *IEEE Transactions on Signal Processing* 67 (11): 3001–12. https://doi.org/10.1109/TSP.2019.2907260.

———. 2020. "Deep Reinforcement Learning for Trading." *The Journal of Financial Data Science* 2 (2): 25–40. https://doi.org/10.3905/jfds.2020.1.030.

A. Feature Inventory

The tables below catalogue every feature type implemented in the feature registry. The subset used in the main state-space experiment is marked in the **In model** column: indicates all variants are used, while a specific parameter value indicates only that variant is used. The active subset is defined in `src/configs/features/hft_lob_features_causal.yaml`. Feature names match the keys produced by the feature pipeline at runtime. Formulas for the active LOB features are given in Chapter 4.

All features used in the main HFT experiment are z-score normalized with causal running

statistics that are updated chronologically and reset at detected session boundaries, with two exceptions noted below the tables: the cyclical time encodings, which are inherently bounded in $[-1, 1]$, and the runtime position feature, which is already bounded in $[-1, 1]$ by the action space definition.

A.1. LOB Microstructure Features

Table 10: LOB microstructure features implemented in the feature registry. The In model column marks features used in the main state-space experiment; when multiple parameter variants exist, the column specifies which variant is used (e.g., $W = 50$).

Feature name	Group	Param	In model	Literature
`hft_book_pressure`	Imbalance	levels {0, 1, 2}		Cao et al. (2004); Biais et al. (1995)
`hft_order_book_imbalance`	Imbalance	$N = 3$		Cont et al. (2014)
`hft_order_count_imbalance`	Imbalance	levels {0, 1}	level 0	Biais et al. (1995)
`hft_microprice`	Fair value	—		Stoikov (2018)
`hft_microprice_divergence`	Fair value	—		Stoikov (2018)
`hft_vwmp_skew`	Fair value	$N = 3$		Stoikov (2018)
`hft_price_vamp`	Fair value	{\$1k, \$5k}		Volume-adjusted mid price
`hft_depth_ratio`	Depth	$N = 4$		Bouchaud et al. (2002)
`hft_spread_bps`	Spread/cost	—		Kyle (1985); Glosten & Milgrom (1985)
`hft_spread_ratio`	Spread/cost	$W \in \{50, 200\}$	$W = 50$	Easley et al. (2012)
`hft_bid_convexity`	Spread/cost	bid side		Bouchaud et al. (2002)
`hft_ask_convexity`	Spread/cost	ask side		Bouchaud et al. (2002)
`hft_bid_slope`	Spread/cost	5 levels		Bouchaud et al. (2002); Kyle (1985)
`hft_ask_slope`	Spread/cost	5 levels		Bouchaud et al. (2002); Kyle (1985)
`hft_ofi`	Order flow	—		Cont et al. (2014)

Feature name	Group	Param	In model	Literature
`hft_ofi_rolling`	Order flow	$W \in \{10, 50, 200\}$	$W = 50$	Cont et al. (2014)
`hft_ofi_multilevel`	Order flow	$N \in \{3, 5\}$		Cont et al. (2014)
`hft_bid_queue_depletion`	Order flow	bid side		Foucault et al. (2005)
`hft_ask_queue_depletion`	Order flow	ask side		Foucault et al. (2005)
`hft_signed_trade_flow`	Order flow	$W \in \{50, 200\}$	$W = 50$	Lee & Ready (1991)
`hft_odd_lot_trade_ratio`	Order flow	$W = 200$		Boehmer et al. (2021)
`hft_odd_lot_imbalance`	Order flow	$W = 200$		Boehmer et al. (2021)
`hft_vpin`	Order flow	$W \in \{100, 500\}$		Easley et al. (2012)
`hft_cancel_to_trade`	Order flow	$W \in \{200, 1000\}$		Foucault et al. (2005)
`hft_trade_arrival_rate`	Order flow	$W \in \{100, 500\}$		Engle (2000)
`hft_large_trade_ratio`	Order flow	$W = 200$		Institutional flow proxy
`hft_ofi_autocorr`	Order flow	$W \in \{50, 200\}$		Cont et al. (2014)
`hft_inter_event_time`	Regime/temp.	—		Engle (2000)
`hft_mid_price_acceleration`	Regime/temp.	—		Abergel et al. (2016)
`hft_hour_sin`	Regime/temp.	—		Wood et al. (1985)
`hft_hour_cos`	Regime/temp.	—		Wood et al. (1985)

Literature: the cited work that motivates or introduces the feature; not a claim that the exact implementation follows the cited paper's specification.

B. TD3 Hyperparameters

The table below lists all hyperparameters for the main AAPL state-space experiment. The full configuration file is `src/configs/scenarios/aapl/td3_hft_lob_state_space.yaml`.

Table 11: TD3 hyperparameters for the main AAPL state-space experiment. Full configuration: `src/configs/scenarios/aapl/td3_hft_lob_state_space.yaml`.

Parameter	Value	Notes
Architecture		
Actor hidden dims	[128, 64]	ReLU activations
Critic hidden dims	[128, 64]	ReLU activations
Optimization		
Actor learning rate	0.0001	Adam
Critic learning rate	0.0001	Adam
Actor weight decay	0.000002	L2 regularization
Critic weight decay	0.000002	
Loss function	Smooth L1 (Huber)	
TD3 mechanisms		
Discount factor γ	0.995	
Target update τ	0.001	Soft update
Policy delay	2	Critic updates per actor update
Target policy noise σ	0.2	Added to target action
Noise clip	0.3	
Exploration noise std	0.1	During training only
Training		
Total steps	2,000,000	
Initial random steps	1,000	Warm-up before learning
Frames per batch	200	
Optimisation steps per batch	5	
Mini-batch size	128	
Replay buffer size	100,000	
Data		

Parameter	Value	Notes
Training rows	15,000	Chronological split
Reward type	Differential Sharpe	$\eta = 0.01$
Transaction fee	0.000	Baseline setting
Random seed	42	

C. Experiment Configuration Example

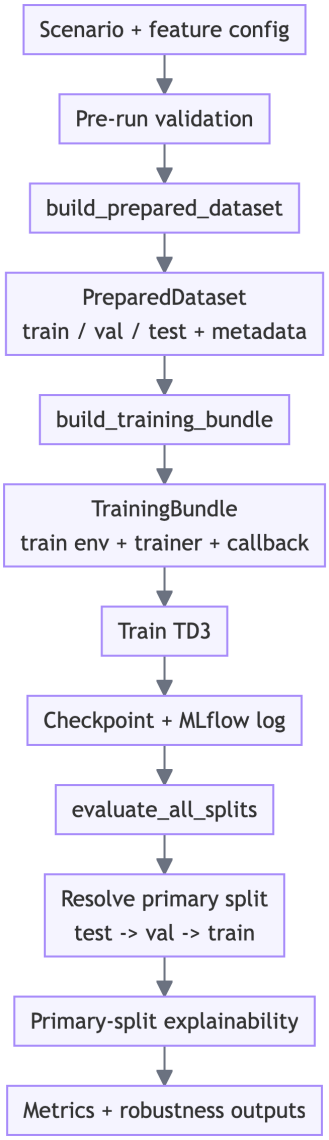
The YAML snippet below is a condensed version of the main experiment configuration, illustrating the configuration-driven workflow described in Chapter 5.

```yaml
seed: 42

data:
  data_path: "stocks/AAPL_2026-02-25_2026-02-27_raw_mbp-10_us_hours.parquet"
  feature_config: "src/configs/features/hft_lob_features_causal.yaml"
  train_size: 15000

env:
  backend: "tradingenv"
  mode: "hft"
  trading_fees: 0.000
  price_column: "close"
  reward_type: "differential_sharpe"
  reward_eta: 0.01

network:
  actor_hidden_dims: [128, 64]
  value_hidden_dims: [128, 64]
```

```yaml
training:
  algorithm: "TD3"
  actor_lr: 0.0001
  value_lr: 0.0001
  gamma: 0.995
  tau: 0.001
  policy_noise: 0.2
  noise_clip: 0.3
  policy_delay: 2
  exploration_noise_std: 0.1
  max_steps: 2000000
  sample_size: 128
  buffer_size: 100000
  loss_function: "smooth_l1"
```

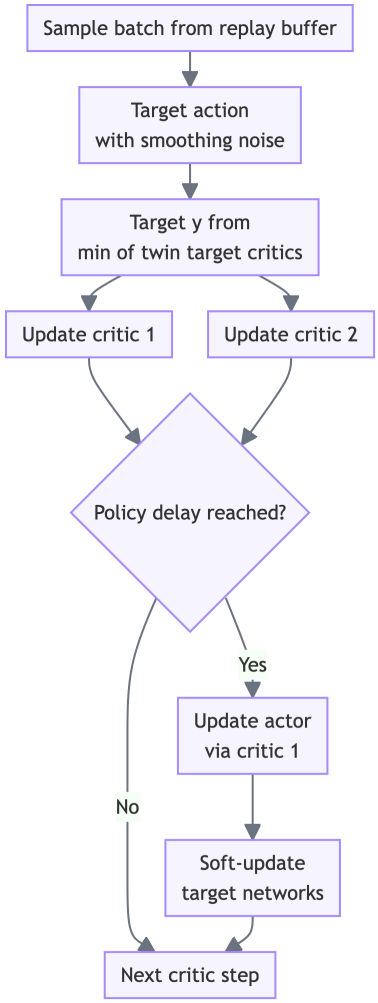
D. TD3 Training Workflow Diagram

The diagram below summarizes the configuration-driven workflow described in Chapter 5.



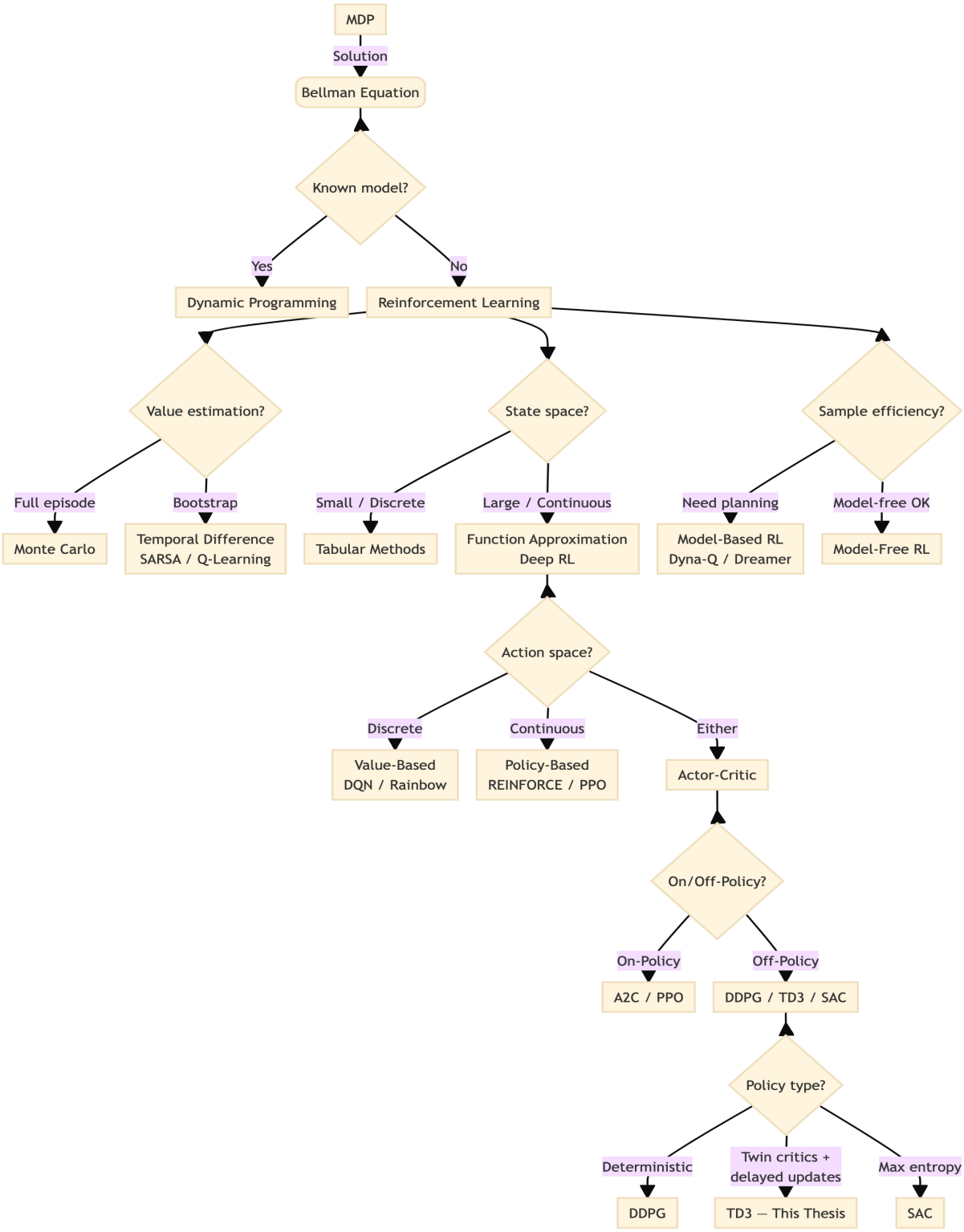
E. TD3 Update Mechanics Diagram

The diagram below summarizes the core TD3 update mechanics used in the thesis implementation, abstracting away from project-specific plumbing.



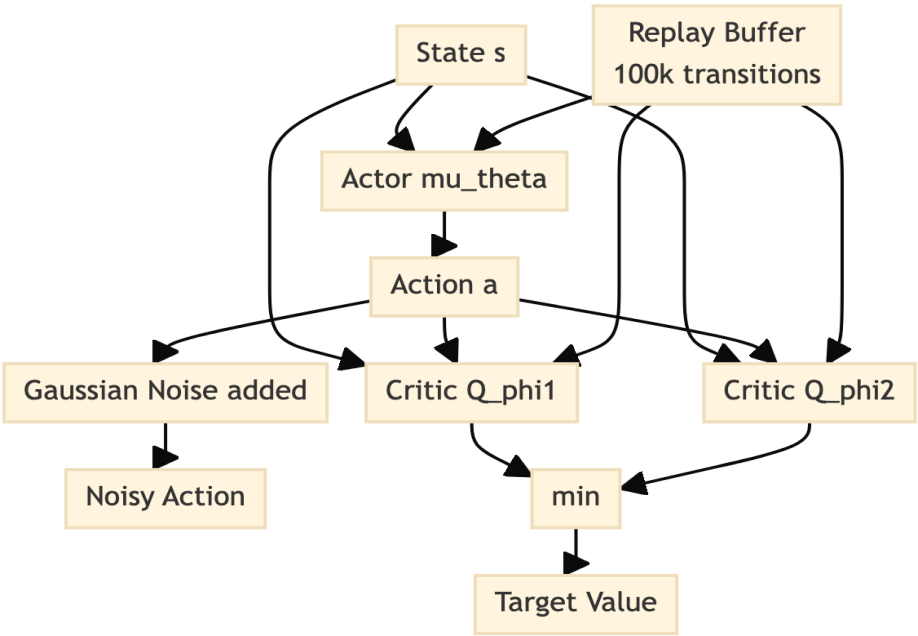
F. RL Algorithm Taxonomy

The diagram below maps the reinforcement learning landscape from the Bellman equation through the key algorithmic relaxations, showing where TD3 sits relative to other families of methods. Source: author's own diagram, based on Sutton and Barto (2018).


MDP
Solution
Bellman Equation
Known model?
Yes
No
Dynamic Programming
Reinforcement Learning
Value estimation?
State space?
Sample efficiency?
Full episode
Bootstrap
Small / Discrete
Large / Continuous
Need planning
Model-free OK
Monte Carlo
Temporal Difference
SARSA / Q-Learning
Tabular Methods
Function Approximation
Deep RL
Model-Based RL
Dyna-Q / Dreamer
Model-Free RL
Action space?
Discrete
Continuous
Either
Value-Based
DQN / Rainbow
Policy-Based
REINFORCE / PPO
Actor-Critic
On/Off-Policy?
On-Policy
Off-Policy
A2C / PPO
DDPG / TD3 / SAC
Policy type?
Deterministic
Twin critics +
delayed updates
Max entropy
DDPG
TD3 — This Thesis
SAC

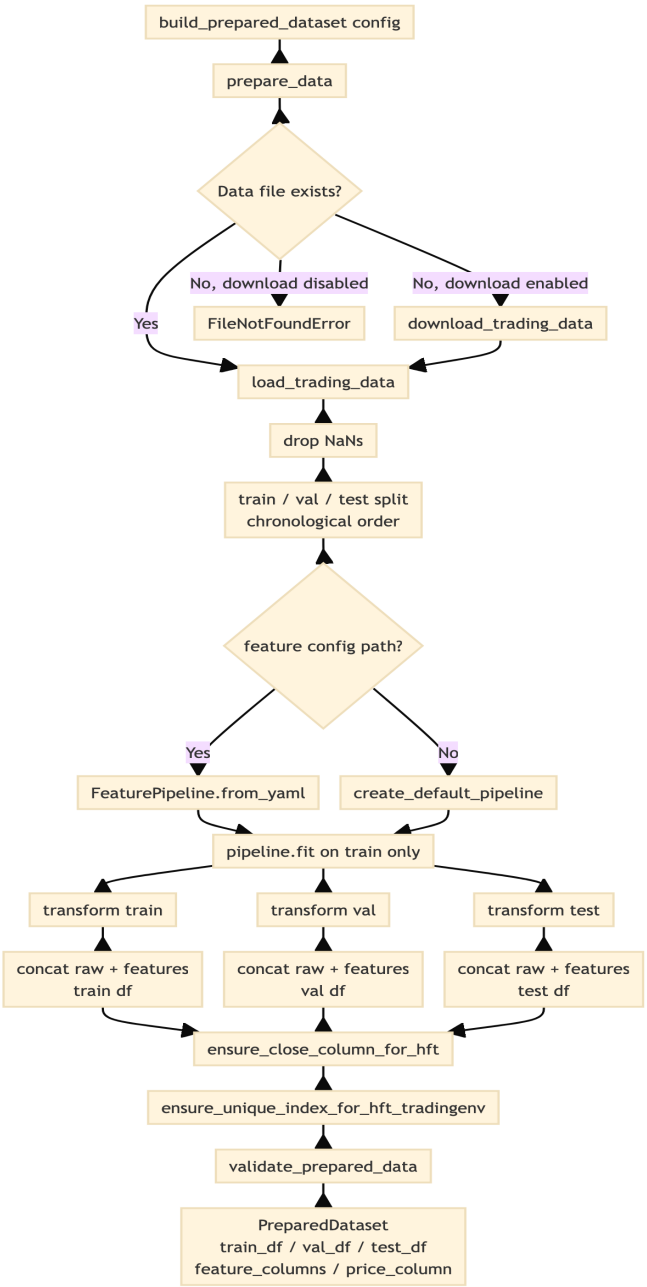
G. TD3 Network Architecture

The diagram below shows how the actor, twin critics, and replay buffer interact in the TD3 architecture used here.



H. Data Preparation Pipeline

The diagram below shows the feature pipeline design that ensures no information from the validation or test splits leaks into the fitted normalization parameters.



I. Offline Feature Selection Pipeline

The feature selection procedure described in Section 4.2 operates in two stages: theory constrains the candidate pool to mechanisms grounded in market microstructure literature; IC/ICIR ranking selects among the available implementations of those mechanisms and removes redundancy via a conditional IC filter. This appendix describes the operational pipeline

that implements Stage 2 of that procedure. It takes a theory-defined candidate pool as input, scores each feature against a forward return proxy, and writes the selected feature specification to a YAML file that the training pipeline loads directly.

I.1. Feature Group Registry

Features are organized into named groups that mirror the state-space design in Section 4.2. Each group corresponds to a distinct microstructure mechanism:

Table 12: Feature groups defined in the group registry (`src/configs/features/feature_groups.yaml`). The imbalance, fair value, spread, and flow groups correspond to the four signal categories in Section 4.2; the regime group provides temporal context.

Group	Description	Features
`imbalance`	Order book imbalance signals	5
`fair_value`	Fair value and microprice estimates	3
`spread`	Spread and cost features	6
`flow`	Order flow and trade flow features	7
`regime`	Regime and temporal features	4

The group registry is a YAML configuration that lists the full specification (name, type, parameters, normalization flag, domain tag) for each feature within each group. Groups can be composed by name, and individual features can be excluded, without editing the feature source code.

I.2. Selection Procedure

The offline selection procedure takes a candidate pool of features, scores each one against a proxy target, and applies a greedy redundancy filter to produce a reduced set. The procedure is run once before training; the resulting feature list is written to a YAML file that the training pipeline loads directly. This ensures that feature selection is not repeated on every training run and that the reduced specification is reproducible.

Step 1: Theory-constrained candidate pool construction. The user specifies which feature groups to include. Each group in `feature_groups.yaml` corresponds to a distinct

microstructure mechanism (see Section 4.2); only groups with a theoretical prior are eligible for inclusion. The `FeatureGroupResolver` expands group names into a flat list of `FeatureConfig` instances. Individual features can be excluded at this stage, but no feature from outside the theory-defined groups can be added. This constraint is enforced by the group registry structure, not by post-hoc filtering.

Step 2: Proxy target. Each feature is scored against a forward log-return target at horizon h:

$$y_t = \log \frac{p_{t+h}}{p_t}$$

This proxy is a cheap, model-free measure of directional predictability; it is not the reward function used during training, but it provides a consistent benchmark for comparing feature informativeness.

Step 3: Scoring. Each feature is evaluated using the Information Coefficient (IC) framework standard in quantitative factor research. The IC of feature f_i at time t is its Spearman rank correlation with the forward return target:

$$\mathrm{IC}_i(t) = \rho_{\mathrm{Sp}}(f_i(t),\, y(t))$$

computed over rolling windows on the validation split. The primary ranking criterion is the Information Coefficient Information Ratio (ICIR):

$$\mathrm{ICIR}_i = \frac{\overline{\mathrm{IC}}_i}{\sigma(\mathrm{IC}_i)}$$

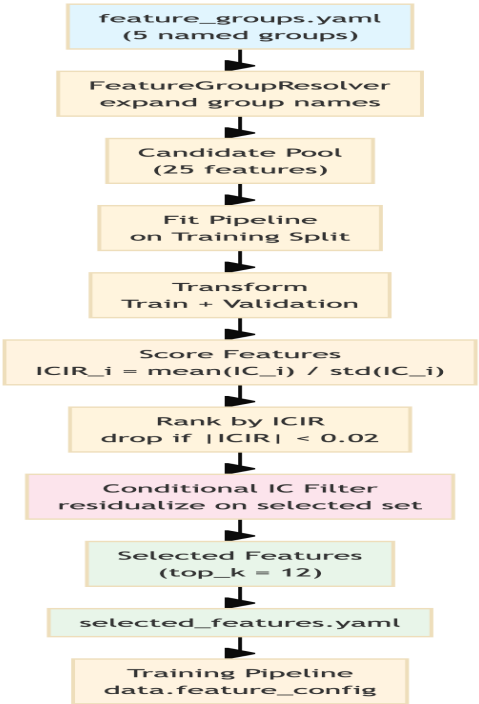
where $\overline{\mathrm{IC}}_i$ is the mean IC and $\sigma(\mathrm{IC}_i)$ is its standard deviation. ICIR captures both signal strength (the numerator) and signal consistency across time (the denominator); a feature with a high mean IC but erratic sign is penalized relative to one with a modest but stable IC. Features with $|\mathrm{ICIR}_i| < 0.02$ are removed before the redundancy step. The t-statistic $t_i = \overline{\mathrm{IC}}_i/(\sigma(\mathrm{IC}_i)/\sqrt{n})$ is reported alongside ICIR as a measure of statistical significance.

Step 4: Redundancy filter. Features are ranked by ICIR. Starting from the highest-ranked, each candidate's residual signal is evaluated: the linear component attributable to already-selected features is regressed out of the candidate, and its ICIR is recomputed on the residuals. A candidate is included in the selected set only if it retains meaningful incremental predictability after conditioning on the features already chosen. This conditional IC approach

is a principled alternative to pairwise correlation thresholding: two features can be highly correlated yet both informative if each predicts a distinct component of the forward return.

Step 5: Output. The selected feature specifications are written to a YAML file that can be referenced directly by the training scenario configuration via `data.feature_config`. The training pipeline reads this file at runtime; no selection logic runs during training.

The diagram below summarizes the offline feature selection pipeline.



The offline nature of this pipeline is deliberate: the proxy target and scoring metrics are not the reward function that the agent optimizes during training. Running selection as a separate step ensures that the agent's training loop remains uncontaminated by any pre-training statistical analysis that could introduce look-ahead bias or circular reasoning.

Glossary of Abbreviations and Key Terms

This glossary defines abbreviations and domain-specific terms used throughout the thesis. Terms are grouped by domain and listed alphabetically within each group.

I.2.1. Market Microstructure and Trading

Term	Definition
Adverse selection	The risk that a counterparty in a trade possesses superior private information, causing uninformed participants (e.g., market makers) to trade at a disadvantage.
Ask slope	Price elasticity of the ask side of the order book across multiple depth levels; a proxy for market impact per unit of sell-side volume.
Basis points (bps)	One hundredth of a percentage point (0.01%). Used to express spreads and returns at fine granularity.
Bid-ask spread	The difference between the lowest ask price and the highest bid price in the order book; the primary observable measure of transaction cost and adverse selection risk.
Bid slope	Price elasticity of the bid side of the order book across multiple depth levels; a proxy for market impact per unit of buy-side volume.
Book convexity	A measure of curvature in the price schedule of the order book, capturing non-linear spacing between price levels on each side.
CumDelta (Cumulative Delta)	Signed trade flow aggregated over a rolling window; the net difference between buyer-initiated and seller-initiated executed volume.
Depth imbalance	Asymmetry in resting volume between the bid and ask sides of the order book at one or more price levels.
ETF	Exchange-Traded Fund. A publicly traded fund that tracks an index, sector, or asset class.
HFT	High-Frequency Trading. Algorithmic trading operating at tick or sub-second timescales, relying on order-book data and microstructure signals rather than price bars.
Inter-event time	The elapsed time between consecutive limit order book updates; a proxy for market activity intensity.

Term	Definition
Kyle's lambda (λ)	A measure of price impact introduced by Kyle (1985): the sensitivity of price changes to net order flow. Higher λ implies lower market liquidity.
LOB	Limit Order Book. The electronic record of all outstanding buy (bid) and sell (ask) limit orders at each price level, maintained by an exchange.
Market maker	A liquidity provider that continuously posts limit orders on both sides of the book, profiting from the bid-ask spread while bearing inventory and adverse selection risk.
MFT	Medium-Frequency Trading. Trading strategies operating at intraday bar (minute to hour) timescales, as distinct from high-frequency tick-level strategies.
Microprice	A volume-weighted estimate of the next transaction price, derived from top-of-book bid and ask quantities (Stoikov 2018). Adjusts the mid-price for queue imbalance.
MicroDiv (Microprice Divergence)	The difference between the microprice and the mid-price; a stationary signal capturing the directional imbalance embedded in the top-of-book queues.
Mid-price	The arithmetic average of the best bid and best ask prices; commonly used as a reference fair value when the spread is symmetric.
OBI (Order Book Imbalance)	Normalized difference in resting volume between bid and ask sides, aggregated across multiple depth levels.
OCI (Order Count Imbalance)	Normalized difference in the number of resting orders between bid and ask sides at a given price level; sensitive to order fragmentation masked by volume-based measures.
OFI (Order Flow Imbalance)	Net directional pressure exerted on the best bid and ask queues by limit order arrivals, cancellations, and executions (Cont, Kukanov, and Stoikov 2014). The empirically dominant contemporaneous predictor of short-term price changes.

Term	Definition
OHLCV	Open, High, Low, Close, Volume. The standard aggregated price bar representation used in low- and medium-frequency trading; does not capture intra-bar queue dynamics.
Odd-lot	A trade smaller than the standard round lot size (typically below 100 shares). Predominantly reflects retail and algorithmic small-lot activity.
Queue depletion	The rate at which the best-bid or best-ask queue is consumed by incoming market orders across consecutive snapshots; a leading indicator of imminent price movement.
RSI	Relative Strength Index. A momentum oscillator measuring the speed and magnitude of price changes; used in rule-based and technical trading strategies.
Spread ratio	The current bid-ask spread divided by its recent rolling mean; a regime signal separating absolute spread level from elevated adverse selection conditions.
VIX	CBOE Volatility Index. A measure of expected 30-day S&P 500 volatility derived from options prices; a common proxy for broad market fear or uncertainty.
VWAP	Volume-Weighted Average Price. The average price of an asset weighted by traded volume over a period; a common execution benchmark.

I.2.2. Reinforcement Learning and Machine Learning

Term	Definition
Actor-critic	A class of reinforcement learning architectures combining a policy network (actor) that selects actions with a value network (critic) that evaluates them. Reduces gradient variance relative to pure policy gradient methods.

Term	Definition
DDPG	Deep Deterministic Policy Gradient (Lillicrap et al. 2016). An off-policy actor-critic algorithm for continuous action spaces using deterministic policies and experience replay. Predecessor to TD3.
DQN	Deep Q-Network (Mnih et al. 2015). A value-based deep RL algorithm combining Q-learning with a deep neural network, experience replay, and a frozen target network. Restricted to discrete action spaces.
DSR	Differential Sharpe Ratio (Moody and Saffell 1998). A reward formulation that approximates the gradient of the Sharpe Ratio with respect to returns, aligning the optimization objective with risk-adjusted performance.
Exploration-exploitation tradeoff	The tension in reinforcement learning between taking actions that yield known rewards (exploitation) and trying new actions to discover potentially better outcomes (exploration). Particularly costly in trading due to transaction costs.
MDP	Markov Decision Process. The formal framework for sequential decision-making under uncertainty, consisting of states, actions, a transition function, and a reward function. Assumes that future states depend only on the current state and action, not on history.
Off-policy	A class of RL algorithms that learn a target policy from data collected by a different behavior policy. Enables experience replay and reuse of historical data. TD3 is off-policy.
On-policy	A class of RL algorithms that must learn from data collected by the same policy being improved. More stable but less sample-efficient than off-policy methods.

Term	Definition
PCA	Principal Component Analysis. A dimensionality reduction technique that projects data onto orthogonal axes of maximum variance.
Policy gradient	A family of RL algorithms that directly optimize the policy parameters through gradient ascent on expected cumulative reward. Handles continuous action spaces naturally.
PPO	Proximal Policy Optimization. An on-policy actor-critic algorithm that constrains policy updates to a trust region to improve stability.
Q-learning	A model-free, off-policy value-based RL algorithm that learns the action-value function $Q(s, a)$ through temporal difference updates (Watkins and Dayan 1992).
Replay buffer	A data structure storing past environment transitions (s_t, a_t, r_t, s_{t+1}) for reuse in off-policy training. Decorrelates training samples and improves sample efficiency.
REINFORCE	A Monte Carlo policy gradient algorithm (Williams 1992) that updates policy parameters using full-episode return estimates. High variance; foundational to modern policy gradient methods.
RL	Reinforcement Learning. A machine learning paradigm in which an agent learns to make sequential decisions by interacting with an environment and maximizing cumulative reward.
SAC	Soft Actor-Critic (Haarnoja et al. 2018). An off-policy actor-critic algorithm incorporating a maximum-entropy objective to encourage exploration through stochastic policies.
SARSA	State-Action-Reward-State-Action. An on-policy temporal difference algorithm that updates $Q(s, a)$ using the action actually taken by the current policy.

Term	Definition
TD3	Twin Delayed Deep Deterministic Policy Gradient (Fujimoto, Hoof, and Meger 2018). An off-policy actor-critic algorithm for continuous control that extends DDPG with clipped double Q-learning, delayed policy updates, and target policy smoothing to reduce overestimation bias. The primary algorithm used in this thesis.
Temporal difference (TD) learning	A class of RL methods that update value estimates based on the difference between successive predictions, without requiring full episode trajectories.
Value function	A function estimating the expected cumulative discounted reward from a given state (or state-action pair) under a given policy. The critic network in actor-critic algorithms approximates this function.

I.2.3. Other

Term	Definition
AAPL	The NASDAQ ticker symbol for Apple Inc. The equity used as the single-asset trading instrument throughout this thesis.
Sharpe ratio	A risk-adjusted performance measure defined as the ratio of mean excess return to return volatility. Higher values indicate better return per unit of risk.